СОДЕРЖАНИЕ

1.4 Компоненты платы TTGO T9 TAudio v1.6 2
1.4.1 MPU9250 3
1.4.2 WM8978 audio codec 5
1.4.3 WS2812 LED String 8
1.5 Среды разработки 10
1.5.1Arduino IDE 10
1.5.2 Visual Studio Code + Плагины 12
1.5.3 ESP-IDF / ESPRESSIVE IDE 13
2 ПРАКТИЧЕСКАЯ ЧАСТЬ 21
2.1 Обработка данных SD карты 21
2.2 Работа с LED Panel 24
2.3 Работа с датчиком MPU9250 26
2.4 Синтезирование звуков с помощью аудиокодека WM8978 28
2.5 Wi-Fi. Режим STA (Станция) 29
2.6 Wi-Fi. Трансляция потокового вещания 34
2.7 Bluetooth. Передача аудиоданных 35
ЗАКЛЮЧЕНИЕ 39
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 40
ПРИЛОЖЕНИЕ А 42
ПРИЛОЖЕНИЕ Б 48
ПРИЛОЖЕНИЕ В 50
ПРИЛОЖЕНИЕ Г 54
ПРИЛОЖЕНИЕ Д 55
ПРИЛОЖЕНИЕ Е 56
ПРИЛОЖЕНИЕ Ж 58

1.1 Компоненты платы TTGO T9 TAudio v1.6

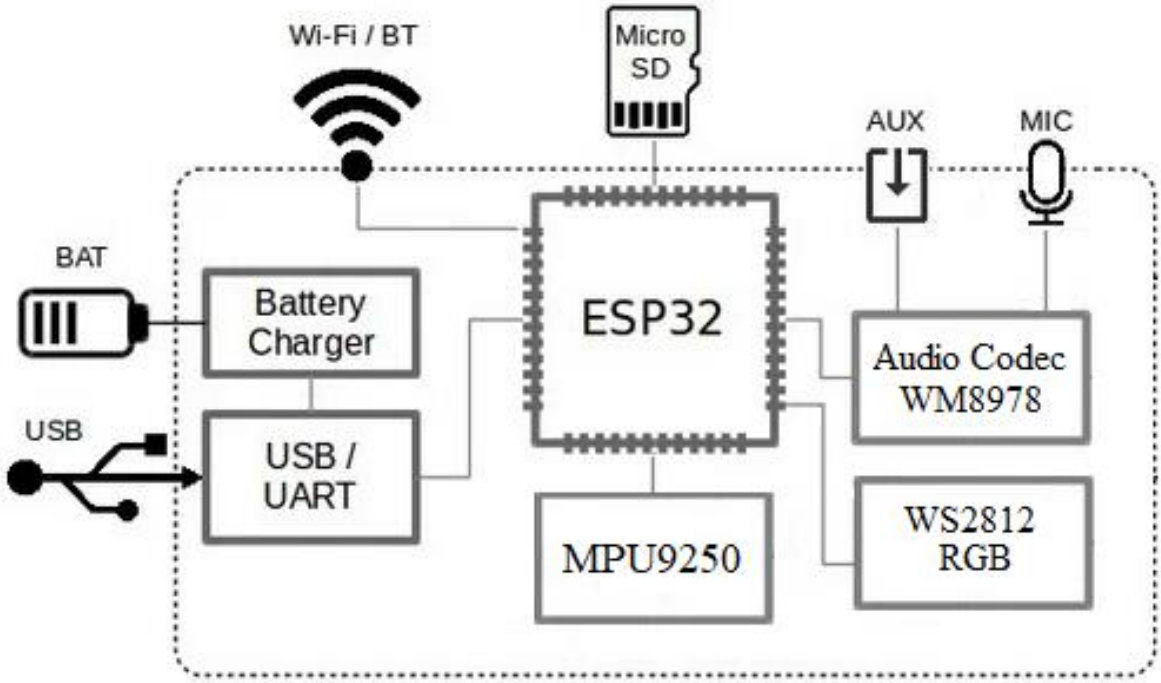

Рисунок 13 - Структурная схема платы TTGO T9 TAudio v1.6

На рисунке 13 изображена структурная схема платы, на которой отображены компоненты устройства.

1.1.1 MPU9250

Датчик MPU9250 реализует функции 3-осевого гироскопа, 3-осевого акселерометра и 3-осевого магнитометра, то есть он представляет собой 9-ти осевой IMU сенсор (Inertial Measurement Unit – инерционное измерительное устройство). Его внешний вид с распиновкой контактов показан на следующем рисунке 14 и таблице 5.

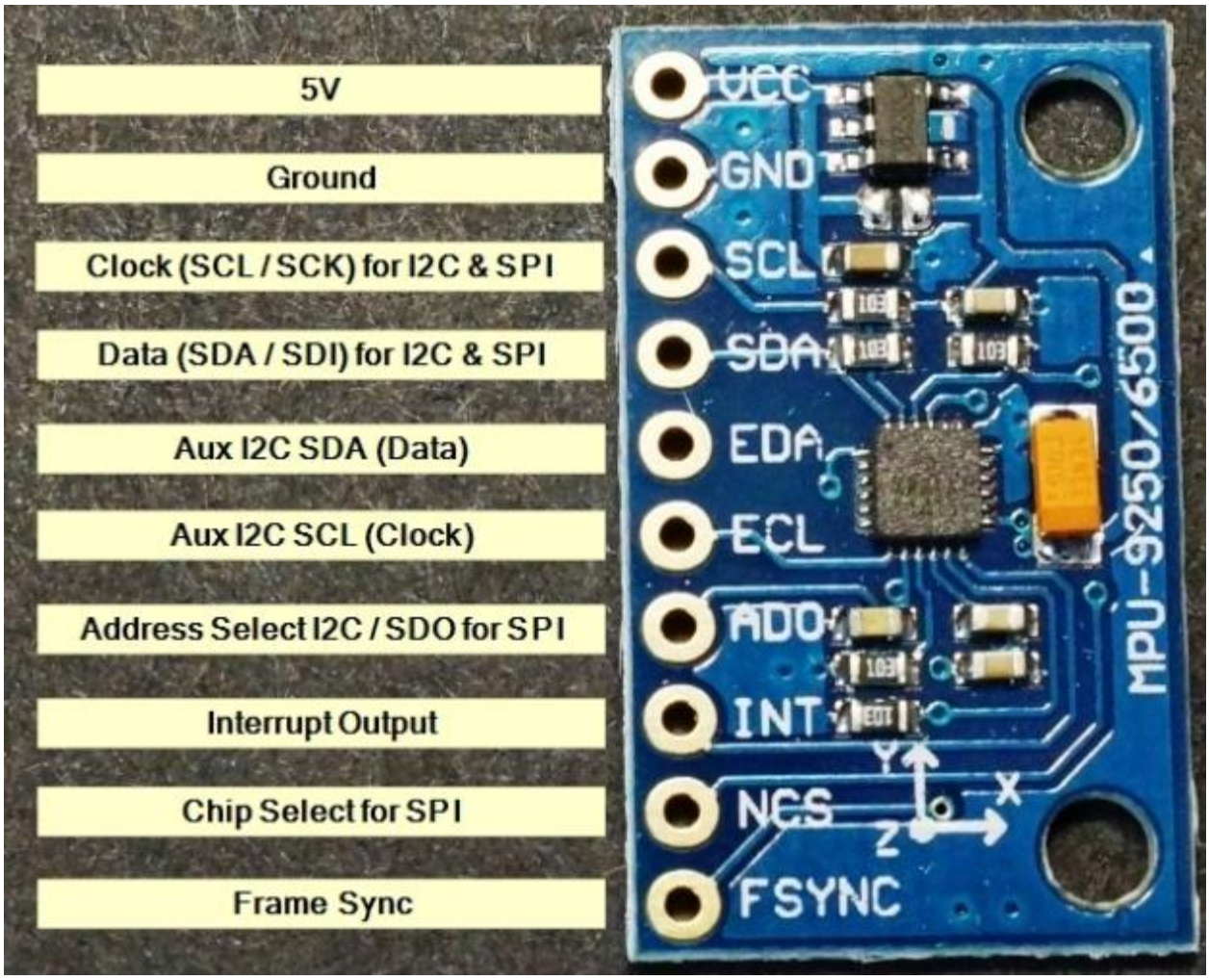

Рисунок 14 – Распиновка датчика MPU9250

ТАБЛИЦА 5. Назначение контактов датчика MPU9250

Обозначение контакта	Назначение контакта
VCC	Внешнее питание 3.3 В
GND	Общий
SCL	Линия тактовых импульсов I^2C и SPI
SDA	Линия данных для I^2C или SPI
EDA	Линия данных при подключении внешних датчиков по шине I^2C
ECL	Линия тактов при подключении внешних датчиков по шине I^2C
AD0	Для выставления адреса I^2C. В режиме SPI это линия данных от датчика
INT	Линия прерываний. Срабатывание настраивается при конфигурировании датчика MPU-9250
NCS	В режиме SPI – линия выбора ведомого (chip select). В режиме I^2C не соединяется ни с чем
FSYNC	Зависит от конфигурации

MPU9250 является самым миниатюрным в мире девятиосевым датчиком. Это говорит о высокой производительности его микросхемы. Состоит корпус датчика из двух мельчайших кристаллов, один из которых отвечает за гироскоп и акселерометр, а другой за магнитометр. Данные с них обрабатываются встроенным сигнальным процессором DMP с помощью алгоритмов Motion Fusion и передаются по интерфейсам I^2C или SPI.

Рассмотрим более подробно работу составных частей датчика.

Гироскоп – это сенсор, реагирующий на изменение углов ориентации в пространстве.

Акселерометр сравнивает проекцию ускорения объекта с гравитационным ускорением и способен замерять линейную скорость объекта, а вкупе с гироскопом – и положение в пространстве.

Магнитометр представляет собой устройство для измерения интенсивности ближайшего магнитного поля, действующего на объект.

Датчик MPU9250 позволяет измерять такие параметры как угол yaw (рыскание), угол pitch (тангаж) и угол roll (крен) [6]. Графическое изображение этих параметров представлено на следующем рисунке 15.

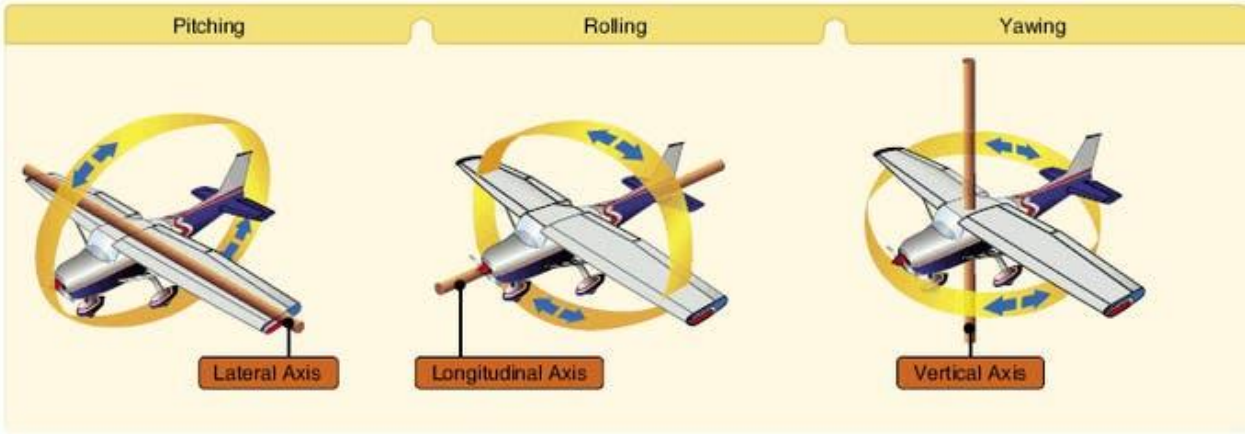

Рисунок 15 – Графическое изображение параметров датчика MPU9250

1.1.2 WM8978 audio codec

WM8978 - это высококачественный стереофонический мультимедийный цифровой сигнальный кодек с низким энергопотреблением. В основном используется в портативных приложениях. Он сочетает в себе предварительное усиление стереофонических дифференциальных микрофонов и привода динамиков, наушников и дифференциальных стерео линейных выходов, уменьшая внешние компоненты, необходимые для таких приложений, как усилители, для которых не требуется отдельный микрофон или наушники.

Расширенные встроенные функции цифровой обработки сигналов, в том числе функция 5-полосной эквалайзера, функция автоматического управления уровнем для смешанного сигнала между АЦП и микрофонным или линейным входом, функция цифрового ограничения записи или воспроизведения. Кроме того, на линии АЦП предусмотрена функция цифровой фильтрации, которая может лучше применять фильтрацию, например «уменьшение шума ветра».

WM8978 может применяться как источник или приёмник. На основе общей эталонной тактовой частоты, такой как 12 МГц и 13 МГц, внутренняя ФАПЧ может обеспечить все необходимые звуковые часы для кодека. На рисунке 389 показана принципиальная схема внутренней структуры чипа WM8978. Фактически каждому переключателю соответствует бит во внутреннем регистре WM8978, и состояние переключателя можно контролировать с помощью регистра управления. Функциональная схема представлена на рисунке 16.

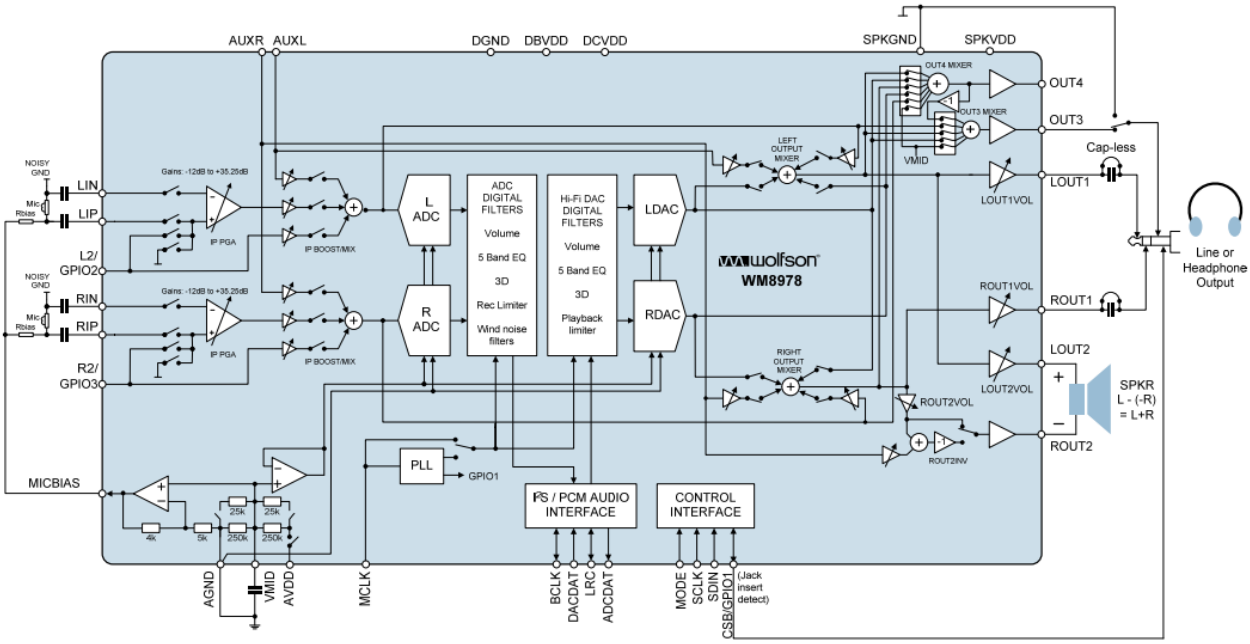

Рисунок 16 – Функциональная схема WM8978

1. Входной раздел

Левая часть структурной схемы WM8978 - это входная часть, которая может использоваться для ввода аналогового звука, то есть для ввода записи. Существует три входных интерфейса: один представляет собой псевдодифференциальный стереомикрофонный вход, состоящий из LIN и LIP, RIN и RIP, другой представляет собой стереомикрофонный вход, состоящий из L2 и R2, и последний представляет собой линейный вход, состоящий из AUXL и AUXR или Используется для передачи входного звукового сигнала.

2. Выходной раздел

Правая часть структурной схемы WM8978 является выходной частью усиления звука. LOUT1 и ROUT1 используются для драйверов наушников. LOUT2 и ROUT2 используются для драйверов динамиков. OUT3 и OUT4 также могут быть сконфигурированы как линейные стерео выходы. OUT4 также может использоваться для обеспечения левого и правого канала Моно микширование.

3. АЦП и ЦАП

Средняя часть структурной схемы WM8978 является основным содержимым чипа, который обрабатывает преобразование звука в АЦП и ЦАП. АЦП обрабатывает входящий звук, включая обработку фильтра АЦП, регулировку громкости, автоматическое управление ограничителем / уровнем входного сигнала и т. Д. Часть ЦАП контролирует эффект вывода звука, включая 5-канальный эквалайзер ЦАП, 3D-усиление ЦАП, ограничение выхода ЦАП и регулировку громкости и т. Д.

4. Интерфейс связи

WM8978 имеет два интерфейса связи, один из которых является цифровым интерфейсом аудиосвязи, а другой - интерфейсом управления. Аудиоинтерфейс использует интерфейс I2S, поддерживающий выравнивание по левому краю, выравнивание по правому краю и стандартные режимы I2S, а также режим DSP A и аналоговый B. Интерфейс управления используется контроллером для отправки команд управления для настройки рабочего состояния WM8978. Он обеспечивает 2-проводный или 3-проводной интерфейс управления. Для контроллера STM32 мы выбираем режим 2-проводного интерфейса, который фактически является режимом шины I2C, а адрес микросхемы фиксируется на 0011010. Внутренние регистры WM8978 могут быть доступны через интерфейс управления для достижения конфигурации рабочей среды микросхемы. Всего имеется 58 регистров, помеченных от R0 до R57. Из-за нехватки места я не буду вдаваться в подробности. Значение каждого регистра см. В разделе «WM8978_v4.5».

Регистр WM8978 имеет длину 16 битов, а старшие 7 бит ([15: 9] битов) используются для маркировки адреса регистра. Следовательно, когда контроллер отправляет управляющие команды в микросхему, он должен передать 16-битную команду, и микросхема будет адресована в соответствии с высоким 7-битным значением принятой команды [7].

1.4.3 WS2812 LED String

На рисунке 17 показан отрезок светодиодной ленты с 8-ю светодиодами WS2812B, включенными последовательно. Каждым из этих светодиодов можно управлять индивидуально по его адресу, используя для этого всего один проводник. Контакт данных первого светодиода IN подключается к микроконтроллеру, а контакт DO/OUT (выходной контакт данных) – к контакту DI (входной контакт данных) следующей светодиодной ленты.

Рисунок 17 – Изображение светодиодной ленты

А на следующем рисунке 18 показано RGB светодиодное кольцо Neopixel с 16 адресуемыми RGB светодиодами, расположенными по кругу. Подобно предыдущему рассмотренному модулю все светодиоды в этом кольце включены последовательно.

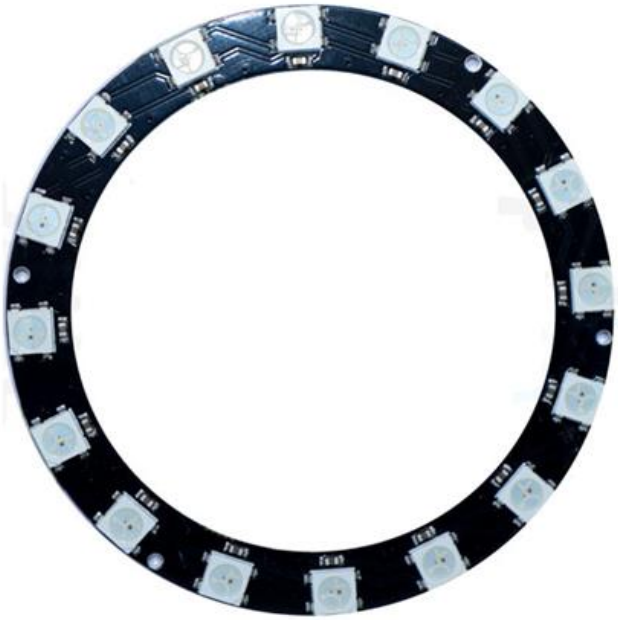

Рисунок 18 – Изображение светодиодного кольца

Светодиод WS2812B содержит 4 контакта, два из которых используются для подачи питания, один для ввода данных и один для вывода данных. Распиновка светодиода WS2812B показана на следующем рисунке 19 [8].

Рисунок 19 – Изображение светодиодной ленты

Назначение контактов следующее:

VCC – обеспечивает подачу питания на модуль, его необходимо подключить к контакту 5V платы Arduino.

GND – общий провод (земля), необходимо подключить к аналогичному контакту платы Arduino.

DI – вход данных от микроконтроллера.

DO – выход данных на следующий модуль или светодиод.

1.2 Среды разработки

1.2.1 Arduino IDE

Программа, написанная в среде Arduino, называется скетч. Скетч пишется в текстовом редакторе, имеющем инструменты вырезки/вставки, поиска/замены текста. Во время сохранения и экспорта проекта в области сообщений появляются пояснения, также могут отображаться возникшие ошибки. Окно вывода текста(консоль) показывает сообщения Arduino, включающие полные отчеты об ошибках и другую информацию. Кнопки панели инструментов позволяют проверить и записать программу, создать, открыть и сохранить скетч, открыть мониторинг последовательной шины [9].

Писать скетчи под микроконтроллер ESP32 можно в среде разработки «Arduino IDE», но перед загрузкой скетча нужно выбрать тип используемой платы из списка меню «Инструменты» > «Плата», а там по умолчанию нет плат ESP32. Вся настройка сводится к тому, что бы в этом списке меню появились эти платы.

В среде «Arduino IDE» имеется «Менеджер плат» который поможет быстро добавить интересующие платы в список. Для этого нужно выполнить следующие действия представленные на рисунке 20:

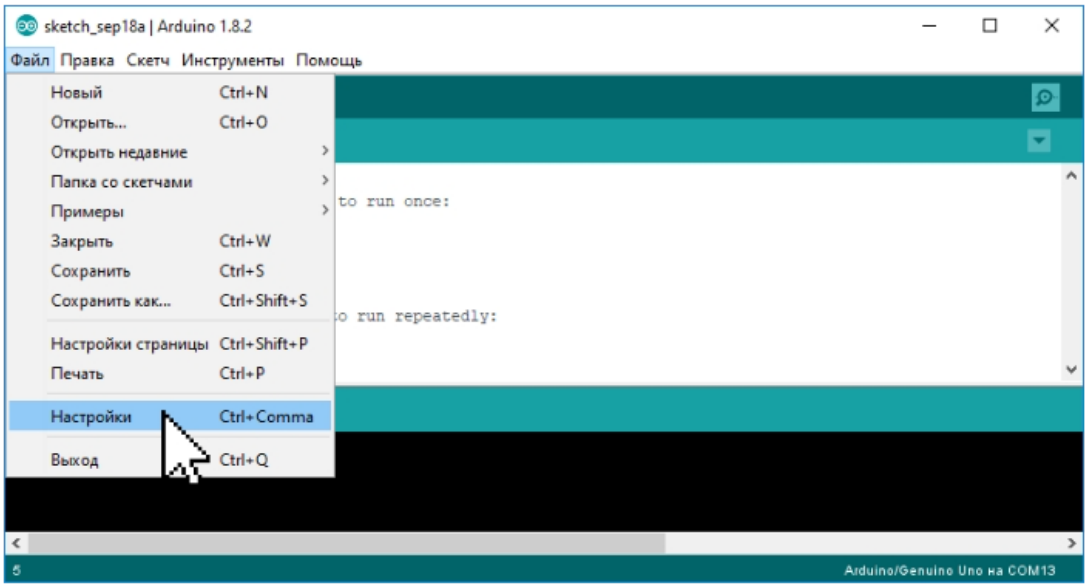

Рисунок 20 – Скриншот окна Arduino IDE

Далее пункт меню: «Файл» > «Настройки» (Рисунок 21)

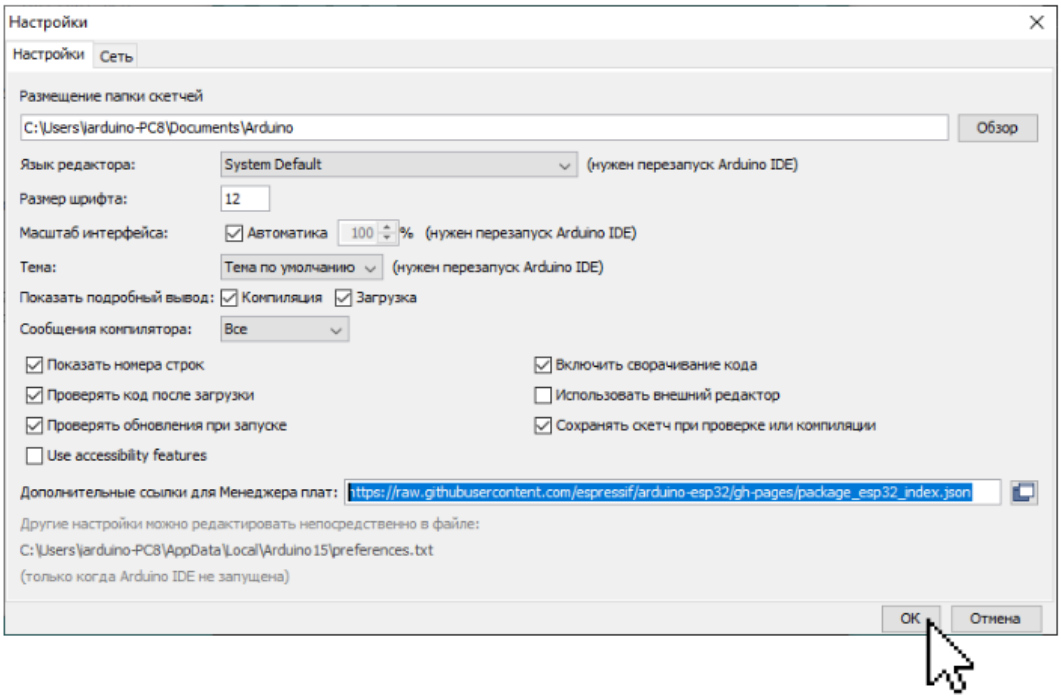

Рисунок 21 – Скриншот окна Arduino IDE

В открывшемся окне заполните поле «Дополнительные ссылки для Менеджера плат» введя в него строку: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json. И нажмём на кнопку «Ok» (Рисунок 222).

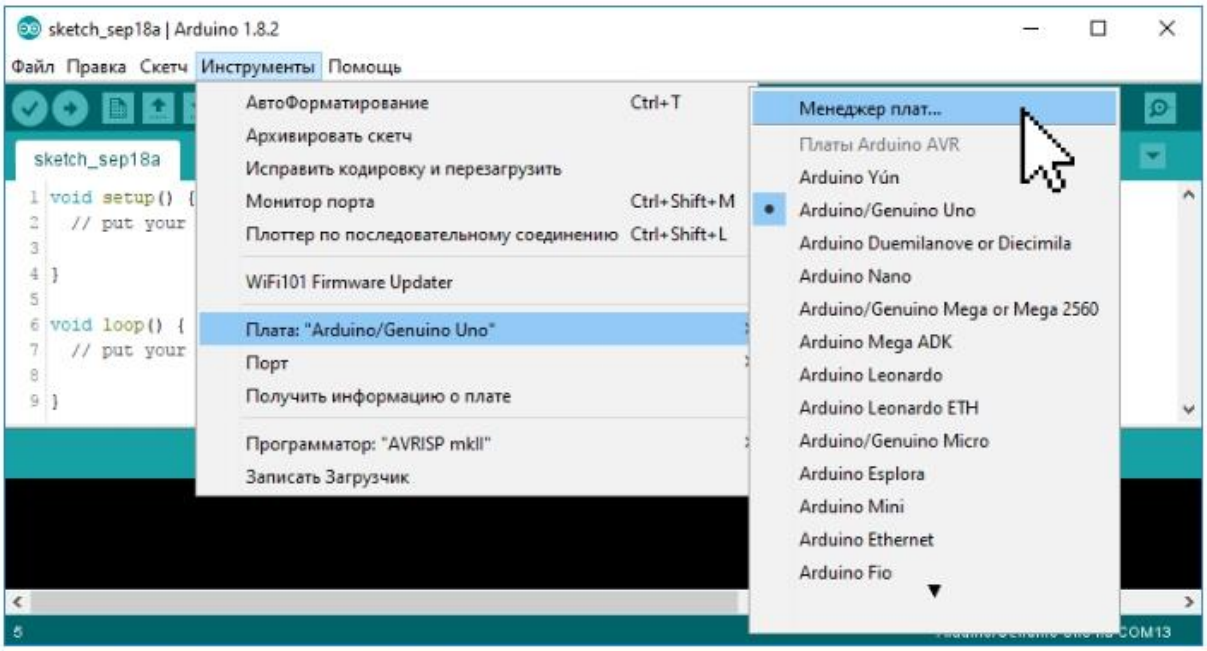

Рисунок 22 – Скриншот окна Arduino IDE

Запускаем «Менеджер плат» выбрав пункт меню «Инструменты» > «Плата» > «Менеджер плат» (Рисунок 23) [10].

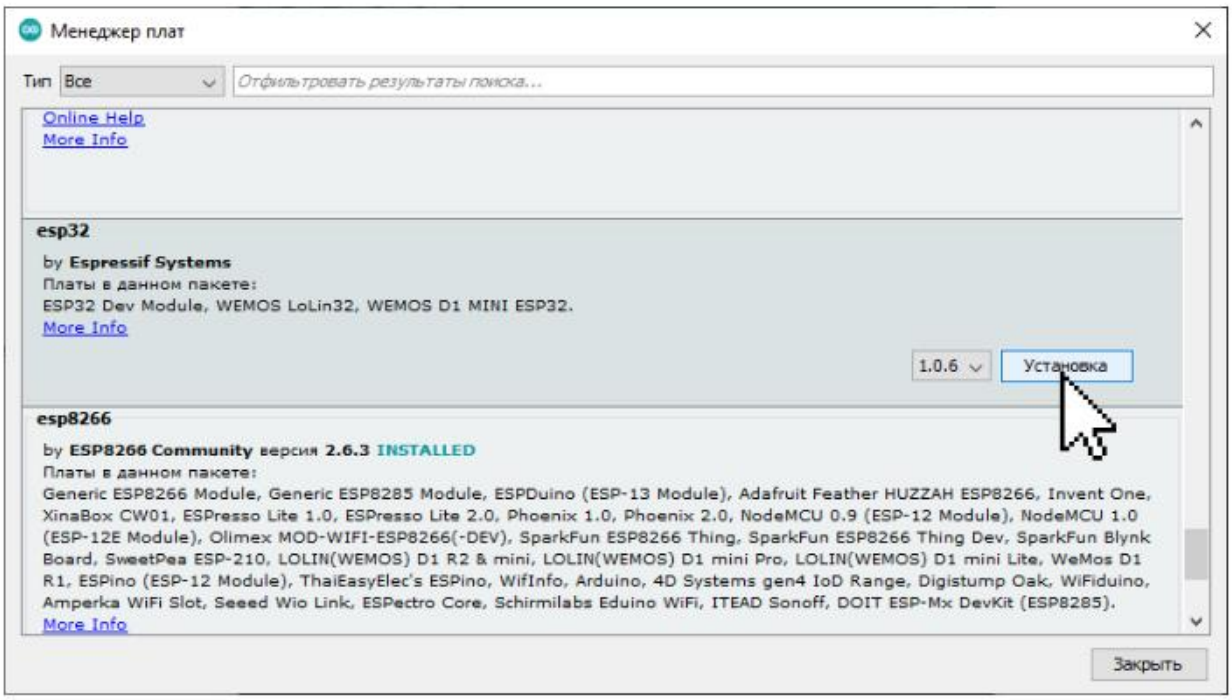

Рисунок 23 – Скриншот окна Arduino IDE

1.2.2 Visual Studio Code + Плагины

Visual Studio Code — редактор исходного кода (IDE, Integrated Development Environment), основан на Electron, разработанный Microsoft для Windows, Linux и macOS. Позиционируется как «лёгкий» редактор кода для кроссплатформенной разработки веб- и облачных приложений. Включает в себя отладчик, инструменты для работы с Git, подсветку синтаксиса, IntelliSense и средства для рефакторинга. Имеет широкие возможности для кастомизации: пользовательские темы, сочетания клавиш и файлы конфигурации. Распространяется бесплатно, разрабатывается как программное обеспечение с открытым исходным кодом. Внешний вид программной среды представлен на рисунке 24.

PlatformIO — это open-source экосистема для разработки. Поддерживает множество платформ, с подробным списком которых можно ознакомиться на официальном сайте (во вкладке Boards можно найти список

всех поддерживаемых устройств), в нем присутствуют все основные популярных микроконтроллеров: Arduino, PIC32, AVR, ESP32, ESP8266, Maixduino, Raspberry Pi и т.п..

PlatformIO может использоваться с VS Code для предоставления мощного набора инструментов, которые помогут в разработке на Ардуино [11] .

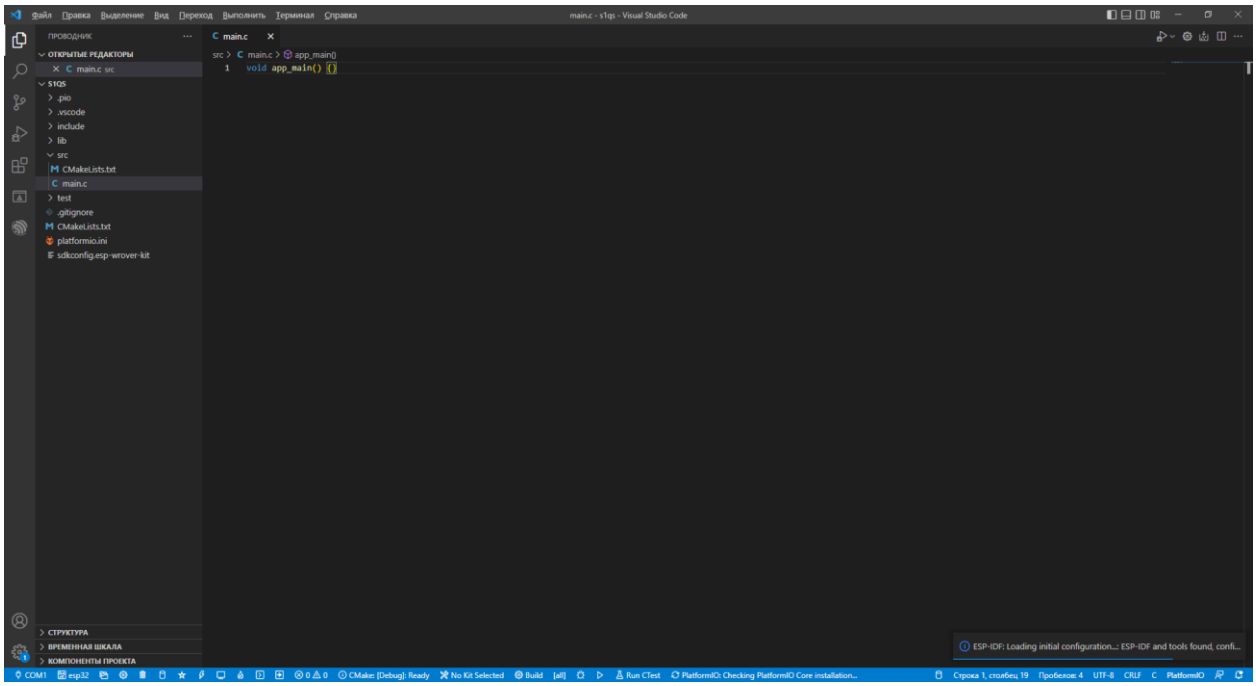

Рисунок 24 – Внешний вид программной среды Visual Studio Code

1.2.3 ESP-IDF / ESPRESSIVE IDE

ESP-IDF позиционируется, как Espressif IoT Development Framework — Фреймворк разработки IoT (Интернет Вещей).

С этим фреймворком легко разрабатывать Bluetooth Low Energy устройства, которыми, можно управлять с мобильного телефона. Подключаться к WiFi и проводной сети Ethernet [12].

Создание проекта в среде выглядит следующим образом:

Откроем дерево проектов (Рисунок 25-26)

Рисунок 25 – Окно среды Espressif IDE

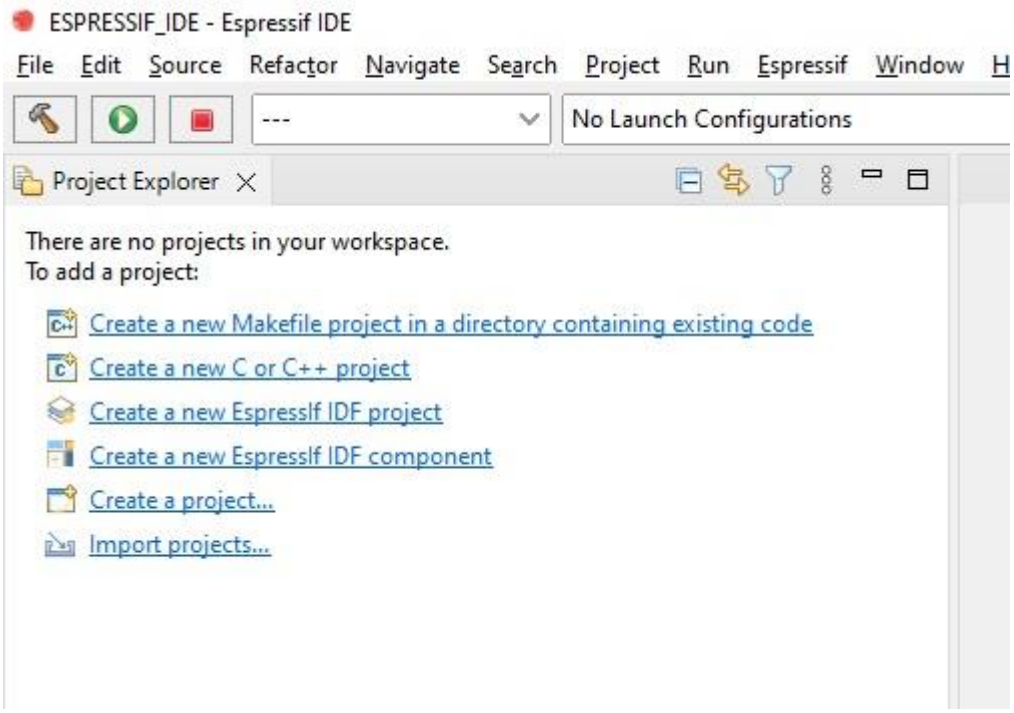

Рисунок 26 – Окно среды Espressif IDE

Прежде чем настроить под себя редактор, давайте попробуем добавить какой-нибудь проект. Скопируем каталог с примером проекта hello_world, находящемуся по пути C:\Espressif\frameworks\esp-idf-v4.4\examples\get-started, в каталог, который мы выбрали для рабочей среды (Рисунок 27-29).

Рисунок 27 – Окно среды Espressif IDE

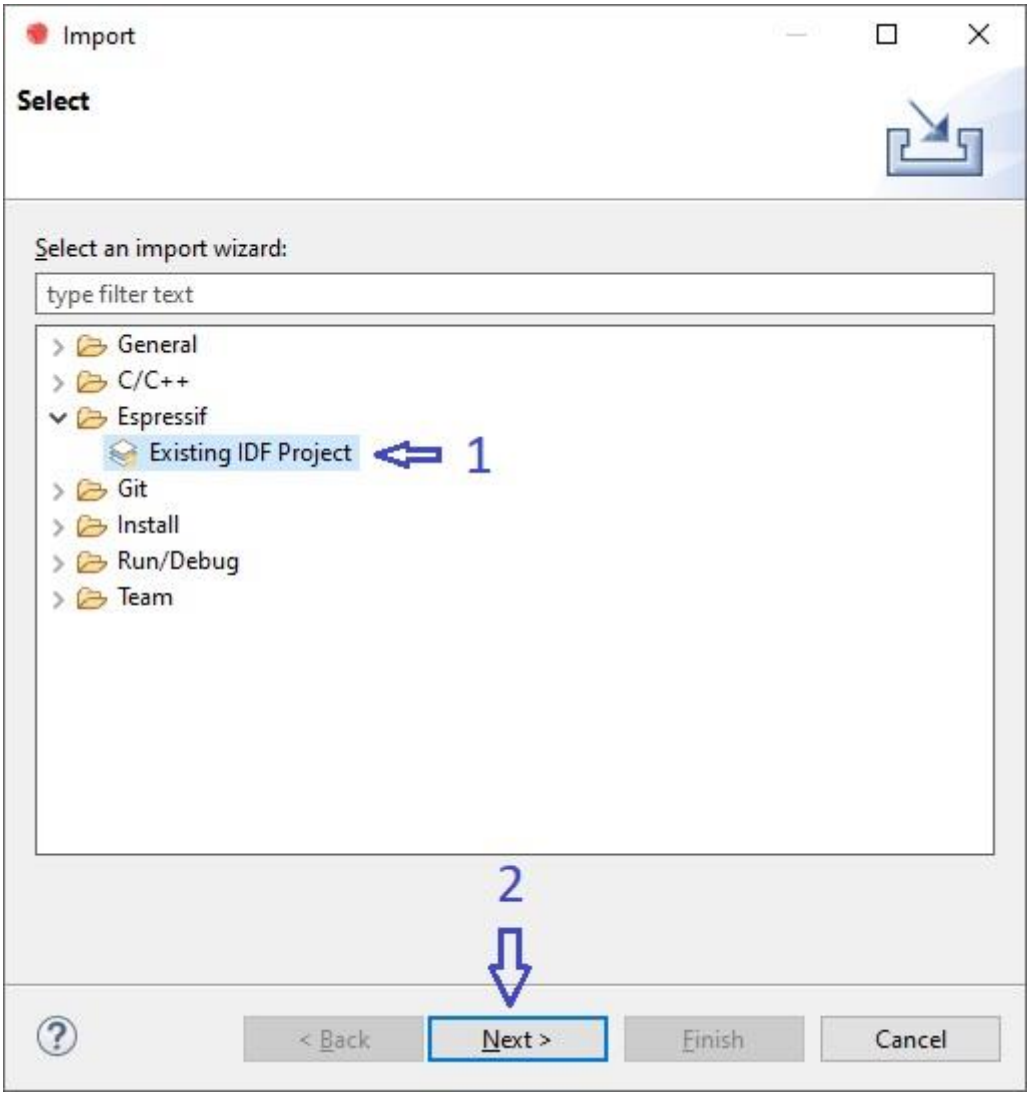

Рисунок 28 – Окно среды Espressif IDE

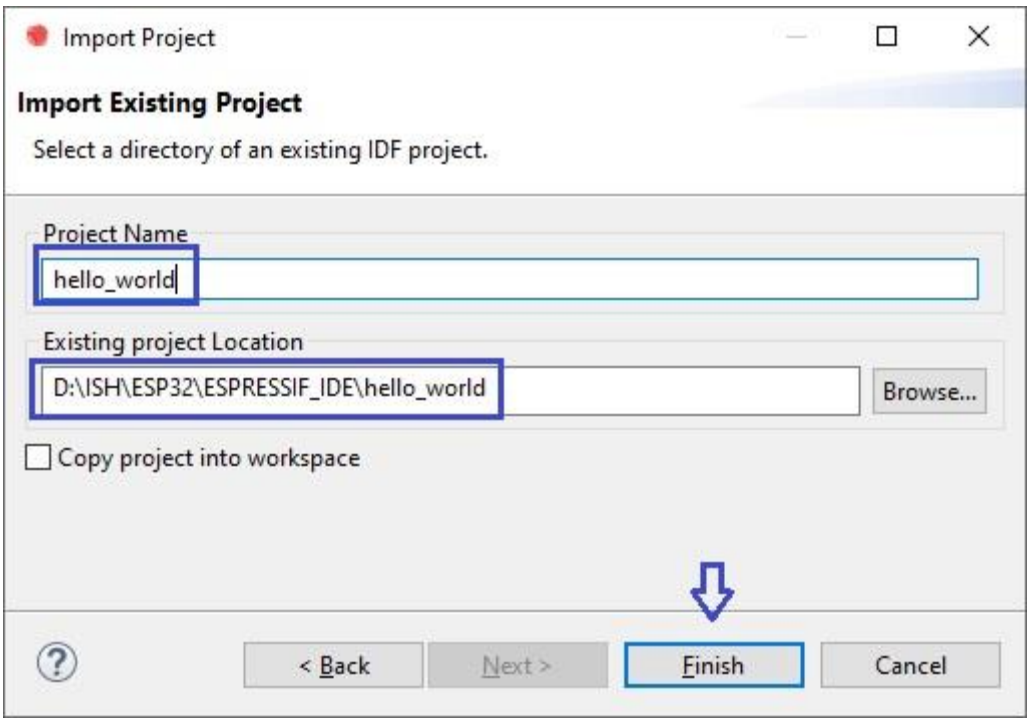

Рисунок 29 – Окно среды Espressif IDE

Попробуем данный проект собрать (Рисунок 30).

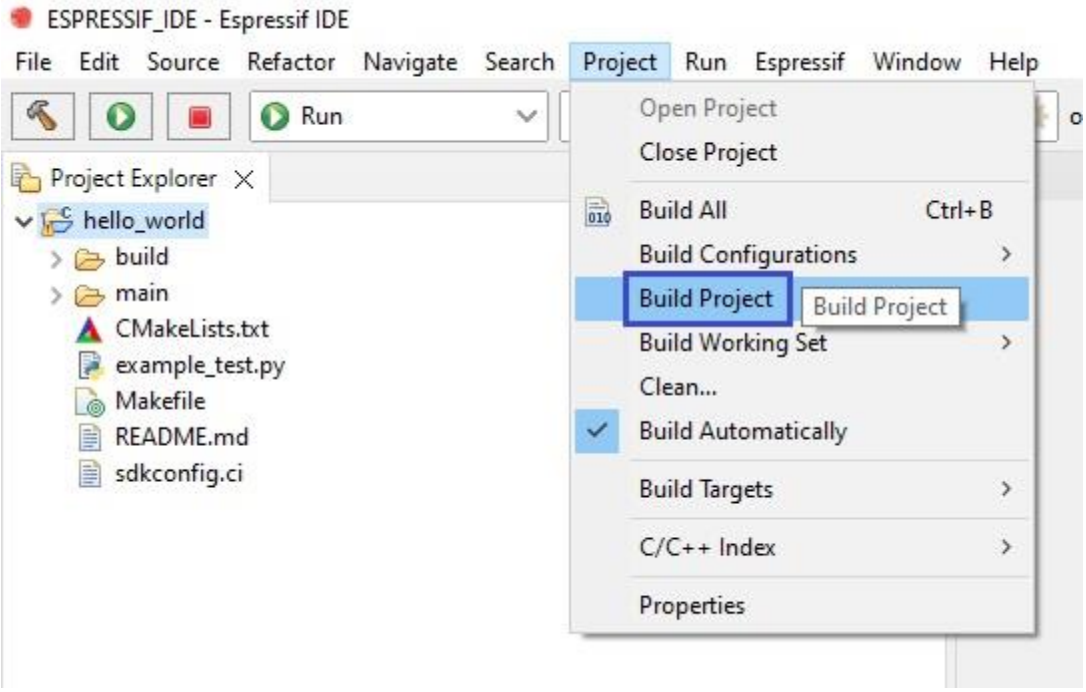

Рисунок 30 – Окно среды Espressif IDE

Начнётся первоначальная сборка компонентов (Рисунок 31).

```
Problems  Tasks  Console ×  Properties  Progress
CDT Build Console [hello_world]
Submodule path 'components/ieee802154/lib': checked out 'f7b5e8059a3bb6f321e79ac3bf2aa4d2a9b93326'
-- Initialising new submodule components/json/cJSON...
Submodule path 'components/json/cJSON': checked out 'd348621ca93571343a56862df7de4ff3bc9b5667'
-- Initialising new submodule components/libsodium/libsodium...
Submodule path 'components/libsodium/libsodium': checked out '4f5e89fa84ce1d178a6765b8b46f2b6f91216677'
-- Initialising new submodule components/lwip/lwip...
Submodule path 'components/lwip/lwip': checked out '2749568fe15df2003f6c3f37f0dfd44f8f01fcd6'
-- Initialising new submodule components/mbedtls/mbedtls...
Submodule path 'components/mbedtls/mbedtls': checked out 'dc406b82b20481bde74a60fa8a776285d3fb9cf3'
-- Initialising new submodule components/mqtt/esp-mqtt...
```

Рисунок 31 – Окно логов среды Espressif IDE

Затем непосредственно сборка проекта (Рисунок 32).

```
Problems  Tasks  Console ×  Properties  Progress
CDT Build Console [hello_world]
[147/1030] ccache C:\Espressif\tools\xtensa-esp32s2-elf\
[148/1030] ccache C:\Espressif\tools\xtensa-esp32s2-elf\
[149/1030] ccache C:\Espressif\tools\xtensa-esp32s2-elf\
[150/1030] ccache C:\Espressif\tools\xtensa-esp32s2-elf\
[151/1030] ccache C:\Espressif\tools\xtensa-esp32s2-elf\
[152/1030] ccache C:\Espressif\tools\xtensa-esp32s2-elf\
[153/1030] ccache C:\Espressif\tools\xtensa-esp32s2-elf\
[154/1030] ccache C:\Espressif\tools\xtensa-esp32s2-elf\
[155/1030] ccache C:\Espressif\tools\xtensa-esp32s2-elf\
[156/1030] ccache C:\Espressif\tools\xtensa-esp32s2-elf\
```

Рисунок 32 – Окно логов среды Espressif IDE

Сборка завершена (Рисунок 33).

```
Problems  Tasks  Console ×  Properties  Progress
CDT Build Console [hello_world]

Used Flash size :  100795 bytes

      .text     :   73135 bytes

      .rodata   :   27404 bytes

Total image size:  151974 bytes (.bin may be padded larger)

C:\Espressif\python_env\idf4.4_py3.8_env\Scripts\python.exe C
Total time taken to build the project: 450 553 ms
```

Рисунок 33 – Окно логов среды Espressif IDE

Это была предварительная сборка. Можно, конечно, данную сборку уже прошить, но тогда конфигурация проекта останется в состоянии «по умолчанию». А если мы хотим сконфигурировать проект по-своему, то после сборки запускаем конфигурацию с помощью двойного клика вот по этому файлу дерева проектов (Рисунок 34).

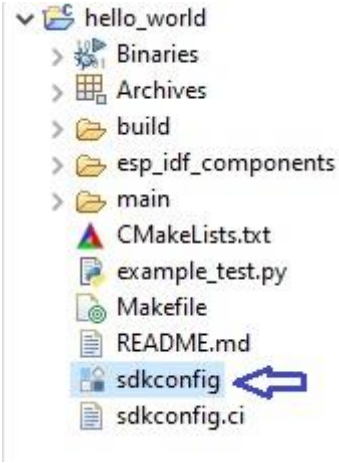

Рисунок 34 – Окно среды Espressif IDE

Запустится конфигуратор в виде дерева (Рисунок 35).

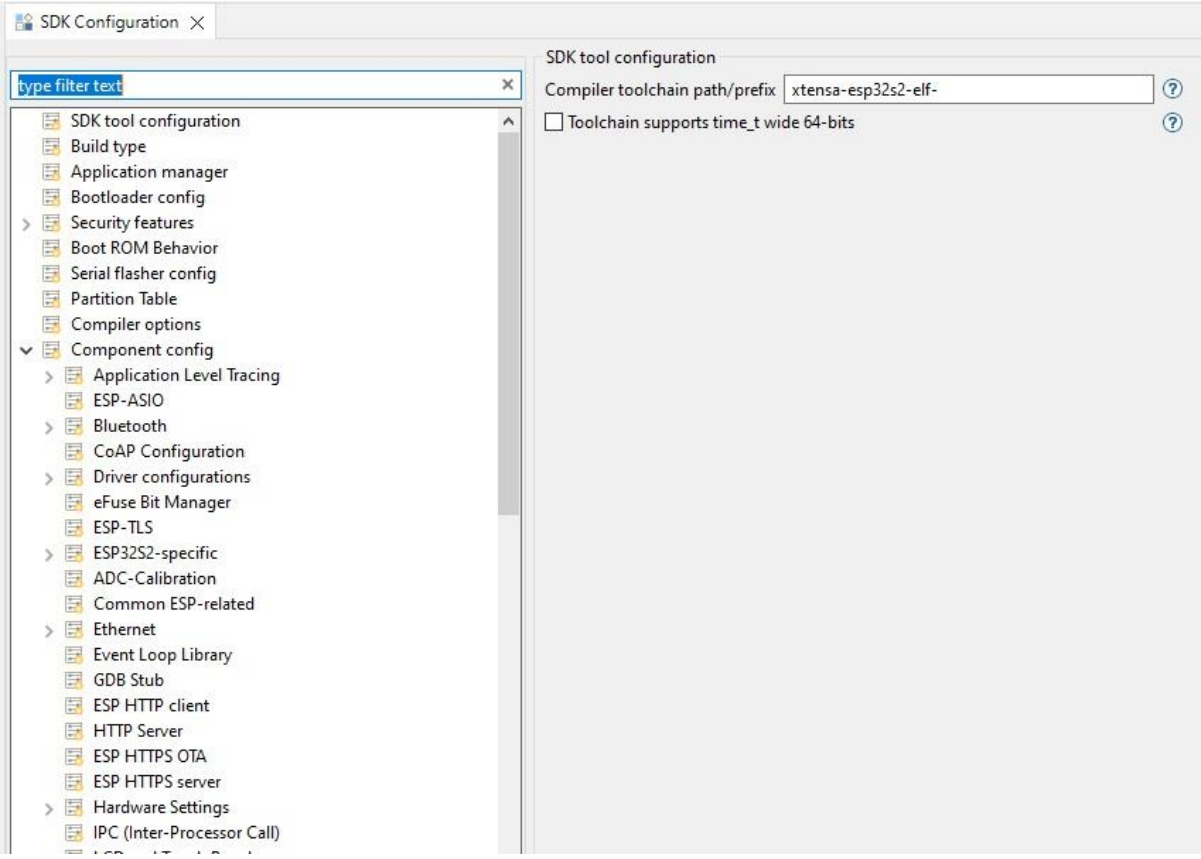

Рисунок 35 – Окно среды Espressif IDE

В данном проекте мы только изменим размер флеш-памяти (Рисунок 36).

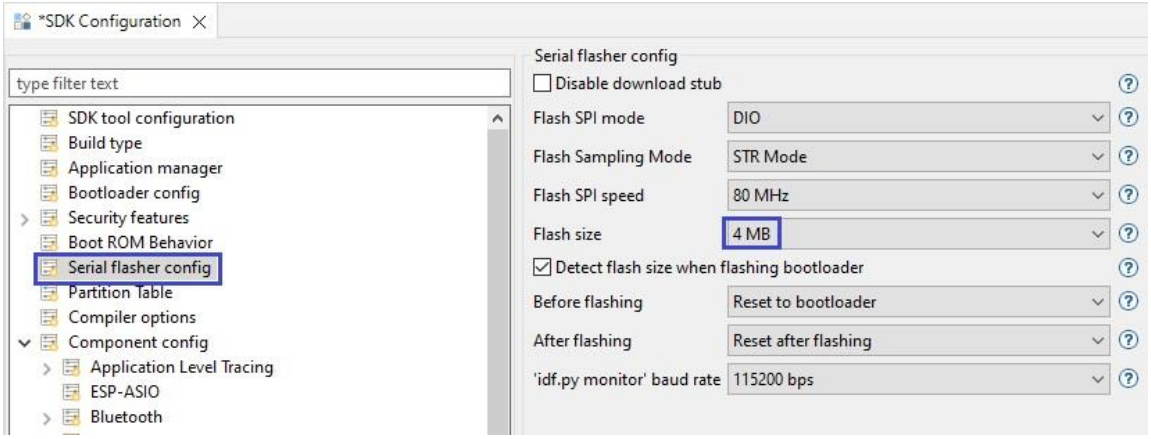

Рисунок 36 – Окно среды Espressif IDE

Закроем вкладку с конфигуратором по крестику, согласившись с его сохранением и ещё раз пересоберём проект таким же образом, как мы его собирали первоначально.

Теперь у нас есть файл прошивки. Но прежде чем его прошить, нужно настроить лаунчер (Launcher) (Рисунок 37).

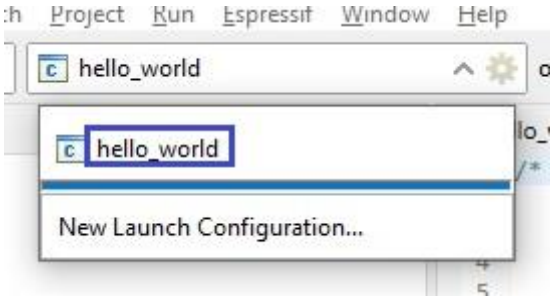

Рисунок 37 – Окно среды Espressif IDE

А здесь останется выбрать правильную цель (Launch target) и выбрать в ней правильный порт (Рисунок 38-39).

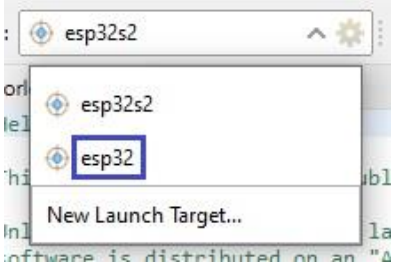

Рисунок 38 – Окно среды Espressif IDE

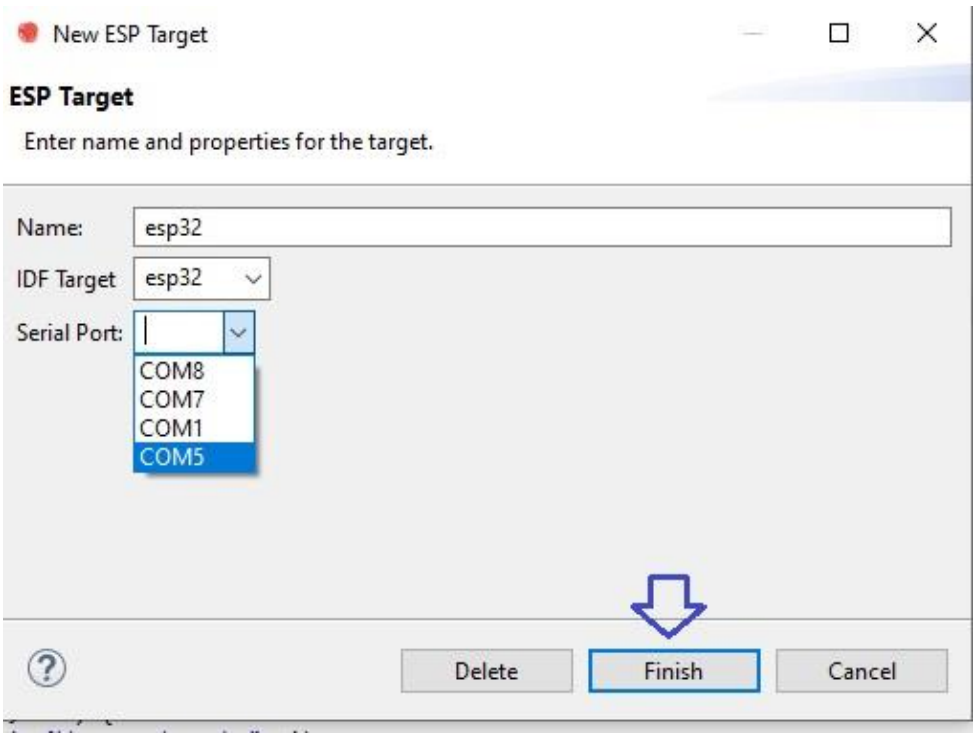

Рисунок 39 – Окно среды Espressif IDE

Попробуем прошить контроллер. Делается это нажатием на кнопку Run (Рисунок 40).

Рисунок 40 – Окно среды Espressif IDE

Если всё нормально, то прошивка должна залиться в контроллер (Рисунок 41).

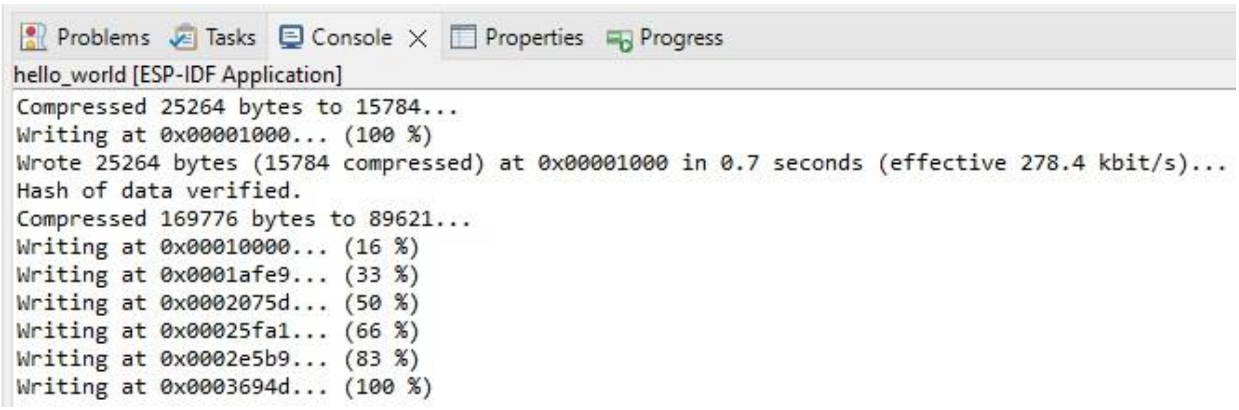

Рисунок 41 – Окно логов среды Espressif IDE

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Обработка данных SD карты

В этом программе демонстрируется, как обрабатывать файлы с карты microSD с помощью ESP32 и Arduino IDE.

Используется пример библиотеки SD, чтобы продемонстрировать все основные функции обработки файлов, включая чтение, удаление, добавление файла и т.д. Схема взаимодействия компонентов представлена на рисунке 42.

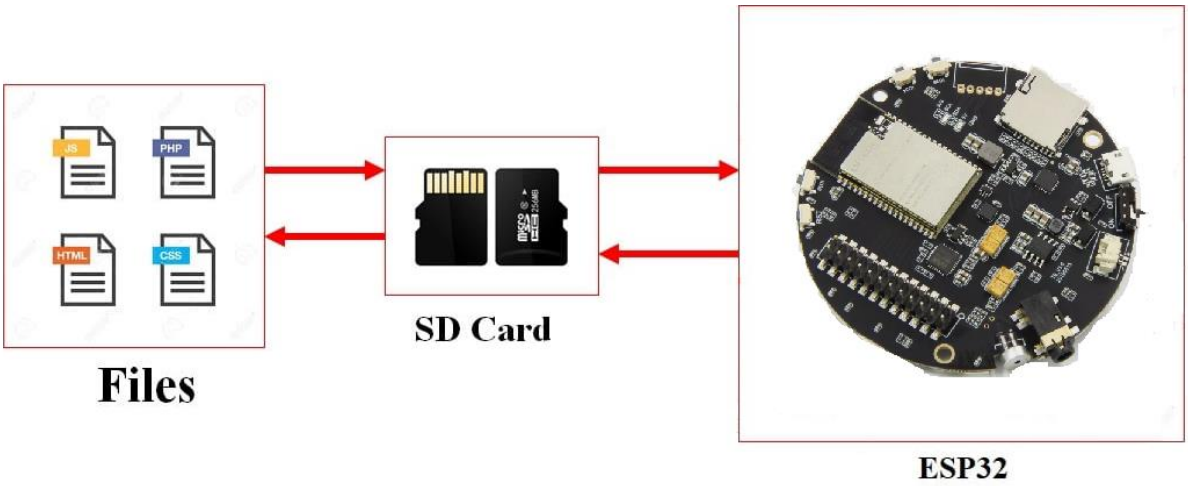

Рисунок 42 – Схема взаимодействия компонентов программы

С помощью данной программы можно продемонстрировать различную обработку файлов на microSD. К ним относятся:

— Перечисление каталога void listDir();

— Создание каталога void createDir();

— Удаление каталога removeDir();

— Чтение файла readFile();

— Запись в файл writeFile();

— Добавление содержимого в конец файла (добавление) appendFile();

— Переименование файла renameFile();

— Удаление файла deleteFile();

— Инициализация карты microSD testFileIO() ;

— Получение размера и типа карты microSD cardType(), cardSize().

Первая функция в примере программы - void listDir(). Она принимает три параметра. “SD", который является файловой системой, имя основного каталога ("/” указывает корневой каталог) и уровень каталога. Сначала файл открывается с помощью fs.open(), и мы указываем имя каталога в качестве аргумента внутри него. Это откроет каталог. Затем, чтобы перечислить все файлы в каталоге, используется root.openNextFile(). Полный текст программы представлен в приложении А.

Пример вывода данных в монитор порта:

```
14:57:03.664 -> Listing directory: /
14:57:03.664 ->   DIR : System Volume Information
14:57:03.664 ->   FILE: IST-942_Ribalko.A.V_lr4.docx  SIZE: 300622
14:57:03.664 -> Creating Dir: /mydir
14:57:03.880 -> Dir created
14:57:03.880 -> Listing directory: /
14:57:03.881 ->   DIR : System Volume Information
14:57:03.881 ->   FILE: IST-942_Ribalko.A.V_lr4.docx  SIZE: 300622
14:57:03.881 -> Removing Dir: /mydir
14:57:03.881 -> Dir removed
14:57:03.881 -> Listing directory: /
14:57:03.881 ->   DIR : System Volume Information
14:57:03.881 -> Listing directory: System Volume Information
14:57:03.881 -> Failed to open directory
14:57:03.881 ->   FILE: IST-942_Ribalko.A.V_lr4.docx  SIZE: 300622
14:57:03.881 -> Writing file: /hello.txt
14:57:03.914 -> File written
14:57:03.914 -> Appending to file: /hello.txt
14:57:03.914 -> Message appended
14:57:03.950 -> Reading file: /hello.txt
```

```
14:57:03.950 -> Read from file: Hello World!
14:57:03.950 -> Deleting file: /foo.txt
14:57:03.950 -> File deleted
14:57:03.950 -> Renaming file /hello.txt to /foo.txt
14:57:03.991 -> File renamed
14:57:03.991 -> Reading file: /foo.txt
14:57:03.991 -> Read from file: Hello World!
14:57:04.677 -> 1048576 bytes read for 683 ms
14:57:05.817 -> 1048576 bytes written for 1151 ms
14:57:05.864 -> Total space: 3802MB
14:57:05.864 -> Used space: 2MB
```

2.2 Работа с LED Panel

В данной программе демонстрируется зажигание RGB светодиодов WS2812 LED String.

Программа реализована на основе функций библиотеки Adafruit_NeoPixel.h. Работа программы выполняется следующим образом:

- Ввод в монитор порта цифры от 1 до 5 для задания режима работы светодиодной ленты;

- Включение светодиодов в зависимости от введённой цифры (1 - Все светодиоды красные; 2 - Все светодиоды зелёные; 3- Все светодиоды синие; 4 - Чередующиеся цвета; 5 - Все светодиоды выключены).

Результат работы программы представлен на рисунке 41,42.

Пример кода программы с выбором режима работы:

```
{
  if (Serial.available() > 0)
  {
      int sym=Serial.parseInt();
       switch (sym)
       {
         case 1:
         colorWipe(strip.Color(0,255,0), 50); // Red
         break;
         case 2:
         colorWipe(strip.Color(255, 0, 0), 50);  // Green
         break;
         case 3:
         colorWipe(strip.Color(0, 0, 255), 50); // BLUE
         break;
         case 4:
         colorWipe(strip.Color(0, 0, 0),1);
         colorWipes(strip.Color(0, 0, 0, 0),1);
```

```
        colorWipes(strip.Color(0, 255, 0),50); // RGB
        break;
        case 5:
        // Чистка
        colorWipe(strip.Color(0, 0, 0),1);
        colorWipes(strip.Color(0, 0, 0, 0),1);
        break;
      }
  }
}
```

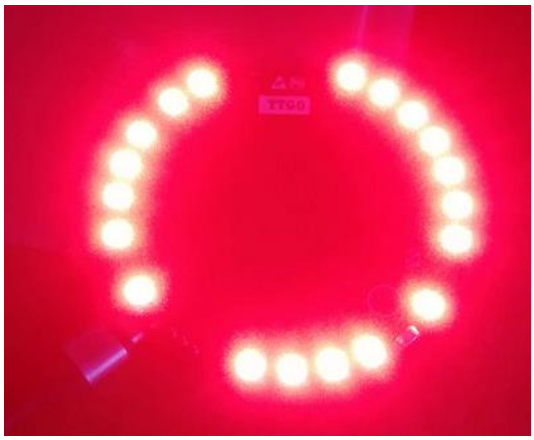

Рисунок 41– Результат работы программы при введение цифры 1 в монитор порта

Рисунок 42– Результат работы программы при введение цифры 4 в монитор порта

2.3 Работа с датчиком MPU9250

В данной программе демонстрируется работа с датчиком MPU9250 .

Программа реализована на основе функций библиотеки MPU9250_asukiaaa.h. Работа программы выполняется следующим образом:

При нажатии на кнопку с адресом GPIO 36 программа переключиться на считывание данных с датчика в режиме Гироскопа;

При нажатии на кнопку с адресом GPIO 34 программа переключиться на считывание данных с датчика в режиме Акселерометра;

При нажатии на кнопку с адресом GPIO 39 программа переключиться на считывание данных с датчика в режиме Магнитометра.

При этом при нажатии одной из кнопок работа другого режима прерывается, т. е одновременно данные считываются в одном режиме.

Пример программы для считывания показаний магнитометра:

```
bool btnState39 = !digitalRead(39);
  if (btnState39 && !flag39) // обработчик нажатия
  {
    flag36 = false;
    flag34 = false;
    Serial.println("stop");
    flag39 = true;
  }
if (!btnState39 && flag39) // обработчик отпускания
  {
 result = mySensor.magUpdate();
  if (result == 0)
  {
    mX = mySensor.magX();
    mY = mySensor.magY();
    mZ = mySensor.magZ();
    mDirection = mySensor.magHorizDirection();
    Serial.println("magX: " + String(mX));
    Serial.println("maxY: " + String(mY));
    Serial.println("magZ: " + String(mZ));
```

```
    Serial.println("horizontal direction: " + String(mDirection));
  }
  else Serial.println("Cannot read mag values " + String(result));
  Serial.println("");
  delay(200);
  }
```

Пример лога программы:

```
17:31:46.728 -> magZ: -53.43
17:31:46.728 -> horizontal direction: 15.21
17:31:46.728 ->
17:31:46.915 -> magX: -107.35
17:31:46.915 -> maxY: 97.05
17:31:46.915 -> magZ: -321.69
17:31:46.915 -> horizontal direction: -47.88
17:31:46.915 ->
17:31:47.131 -> magX: -3359.75
17:31:47.131 -> maxY: 204.76
17:31:47.131 -> magZ: -3091.88
17:31:47.131 -> horizontal direction: -86.51
```

2.4 Синтезирование звуков с помощью аудиокодека WM8978

В данной программе демонстрируется работа с датчиком MPU9250. Программа реализована на основе функций библиотек WM8978.h и FaustSawtooth.h. Работа программы выполняется следующим образом:

Настройка параметров (конфигурация) аудиокодека:

```
wm8978.init();
wm8978.addaCfg(1,1);
wm8978.inputCfg(1,0,0);
wm8978.outputCfg(1,0);
wm8978.micGain(30);
wm8978.auxGain(0);
wm8978.lineinGain(0);
wm8978.spkVolSet(0);
wm8978.hpVolSet(10,10);
wm8978.i2sCfg(2,0);
faustSawtooth.start();
```

Воспроизведение синтезированных звуков:

```
wm8978.hpVolSet(10,10);
faustSawtooth.setParamValue("freq",rand()%(2000-50 + 1) + 50);
delay(1000);
wm8978.hpVolSet(10,0);
faustSawtooth.setParamValue("freq",rand()%(2000-50 + 1) + 50);
delay(1000);
wm8978.hpVolSet(0,10);
faustSawtooth.setParamValue("freq",rand()%(2000-50 + 1) + 50);
delay(1000);
```

2.5 Wi-Fi. Режим STA (Станция)

Режимы работы WiFi контроллера — это станция (или кратко STA) и точка доступа (AP).

Самым простым из этих режимов является первый — режим станции.

Режим станции (STA) — это такой режим, в котором контроллер не создает собственную сеть, а подключается к любой существующей сети Wi-Fi, например, к существующей локальной сети или к другому устройству, работающему в режиме точки доступа (AP).

В данной программе плата будет подключена к существующей точке доступа. В роли точки доступа будет использоваться мобильный телефон в режиме точки доступа.

Для начала введём параметры сети (Рисунок 43) к которой подключаемся и указываем адрес светодиода, который должен загораться при удачном подключении.

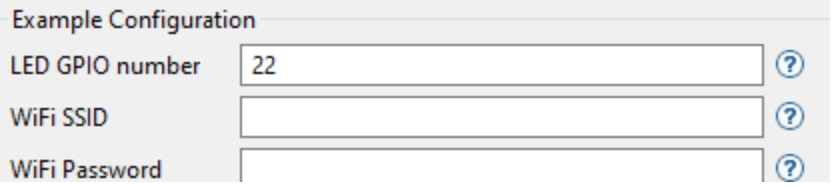

Рисунок 43 – Ввод параметров конфигурации платы

Активируем точку доступа на телефоне

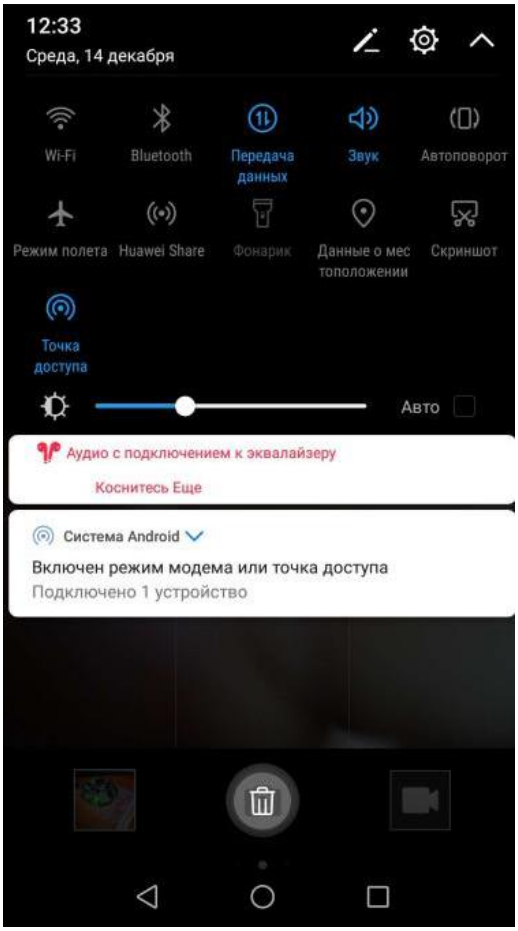

Рисунок 44 – Активация точки доступа в телефоне

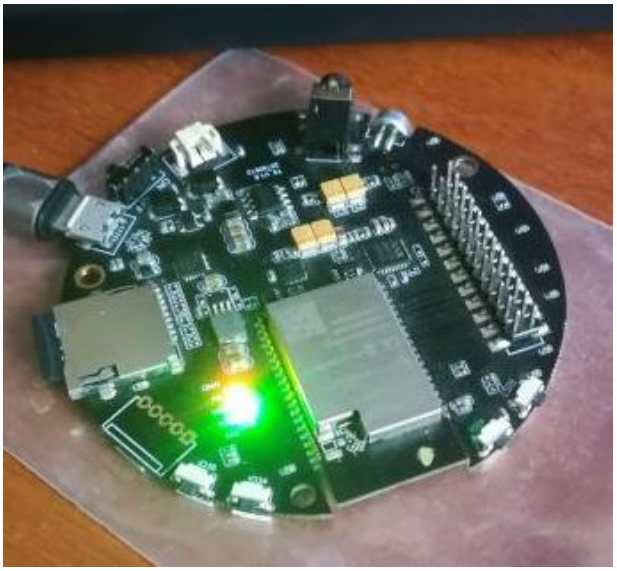

Рисунок 45 – Включение светодиода при подключении к точке доступа

Далее мы создаём UDP-клиент, который будет передавать в сеть на определённый адрес и порт постоянно инкрементирующийся 16-разрядные знаковые величины.

Для проверки работы программы воспользуемся приложением TCP UDP Client Server (Рисунок 46-50):

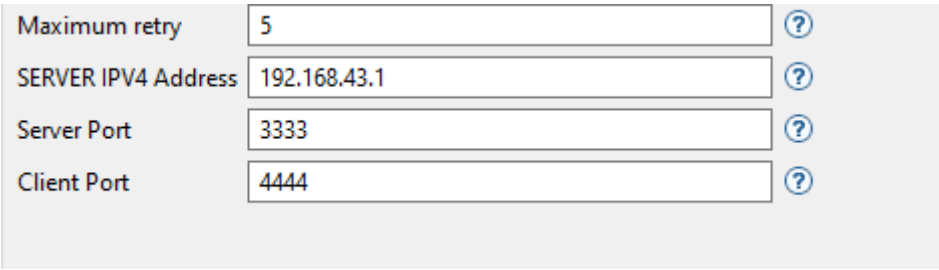

Рисунок 46 - Ввод параметров конфигурации платы

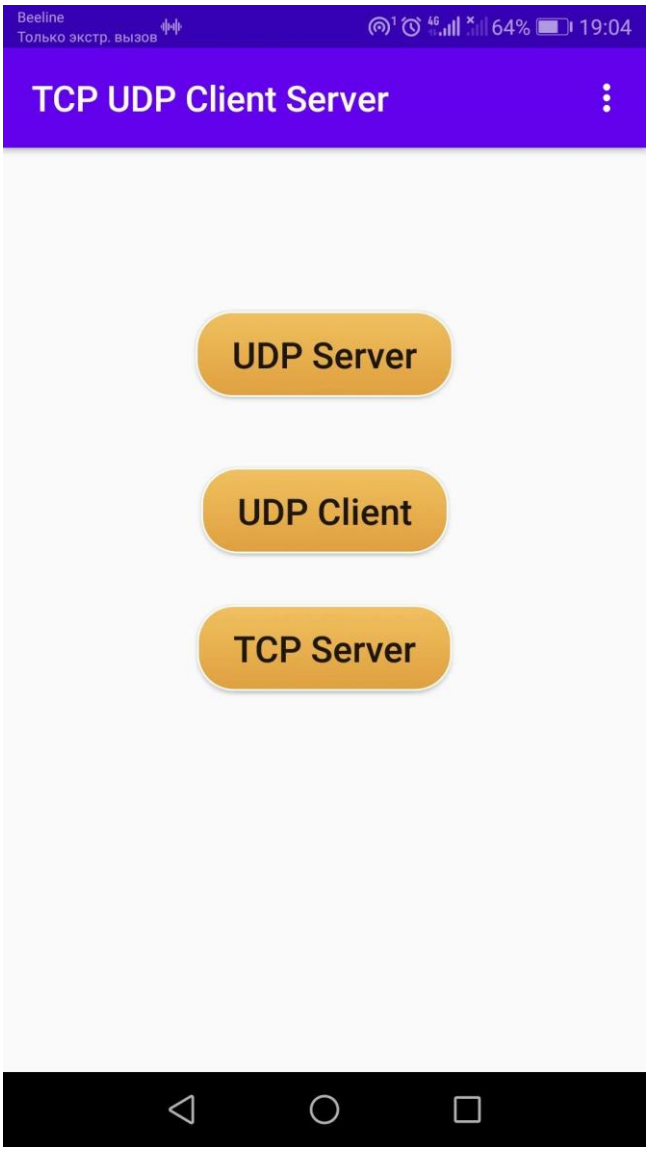

Рисунок 47– Окно приложения TCP UDP Client Server

Нажмём на UDP Server и далее Start.

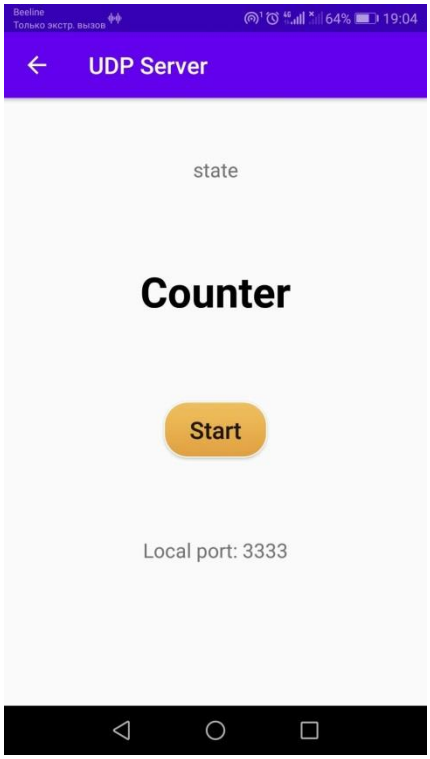

Рисунок 48– Окно приложения TCP UDP Client Server

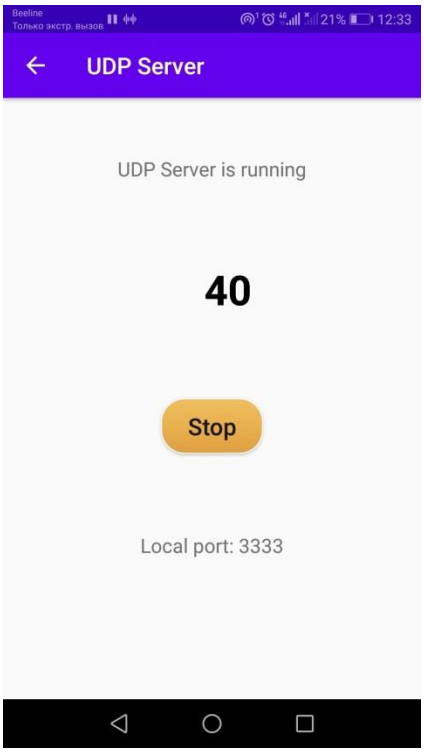

Рисунок 49 – Окно приложения TCP UDP Client Server

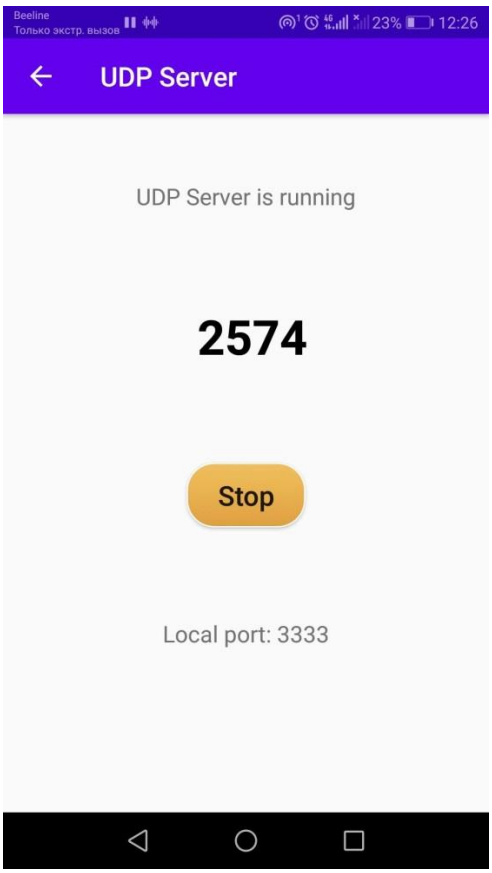

Рисунок 50 – Окно приложения TCP UDP Client Server

Пример логов:

I (1517) esp_netif_handlers: sta ip: 192.168.43.115, mask: 255.255.255.0, gw: 192.168.43.1

I (1517) wifi: got ip:192.168.43.115

I (1517) wifi: connected to ap SSID:Anime

I (1527) udp: Create socket...

I (1527) main: esp_wifi_sta_get_ap_info: 0x0000

I (1527) udp: socket was binded

I (1537) main: SSID: Anime

I (2537) main: esp_wifi_sta_get_ap_info: 0x0000

I (2537) main: SSID: Anime

2.6 Wi-Fi. Трансляция потокового вещания

Программа предназначена для принятия платой ESP32 потокового радиовещания по сети Wi-Fi и передачи данных через аудиокодек на AUX выход, через который можно прослушивать музыку используя наушники или колонки. Работу программы можно представить следующим образом, как представлено на рисунке5 1.

Рисунок 51 – Структурная схема программы

Пример задания параметров к подключаемой точке доступа и конфигурация аудиотрансляции:

```
WiFi.begin("Anime", "12345678");
  while (!WiFi.isConnected())
 {
    delay(10);
  }
  ESP_LOGI(TAG, "Connected. Starting MP3...");
  audio.connecttohost("https://str.pcradio.ru/energy_spb-hi");
  dac.setSPKvol(20); //(Max 63).
  dac.setHPvol(52, 52);// left, right channel.
```

2.7 Bluetooth. Передача аудиоданных

Программа задаёт параметры Bluetooth модуля платы и создаёт точку доступа к ней. При подключении к плате можно использовать её как аудиовыход для воспроизведения музыки. Работу программы можно представить следующим образом, как представлено на рисунке 52.

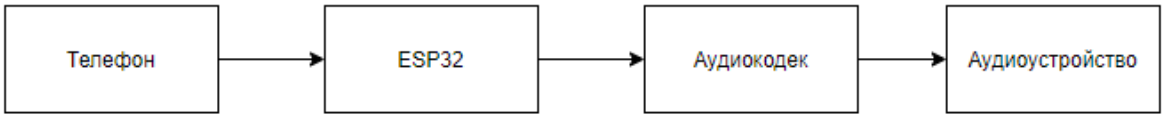

Рисунок 52– Структурная схема программы

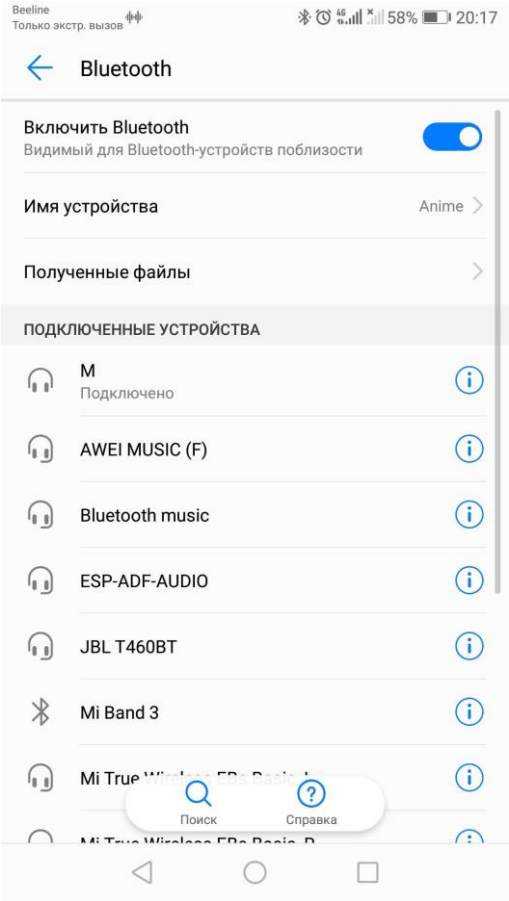

Рисунок 53 – Подключение к плате используя телефон

Пример логов при подключении к плате:

```
W (12520) BT_HCI: hcif conn complete: hdl 0x80, st 0x0

E (12840) BT_APPL: bta_av_rc_create ACP handle exist for shdl:0

I (12840) BT_AV: A2DP connection state: Connecting, [5c:03:39:3e:be:0a]

I (12850) I2S: DMA Malloc info, datalen=blocksize=240, dma_buf_count=6

I (12850) I2S: I2S0, MCLK output by GPIO0

I (12910) BT_AV: A2DP audio stream configuration, codec type: 0

I (12910) BT_AV: Configure audio player: 21-15-2-35

I (12910) BT_AV: Audio player configured, sample rate: 44100

W (12990) BT_APPL: new conn_srvc id:19, app_id:0

I (12990) BT_AV: A2DP connection state: Connected, [5c:03:39:3e:be:0a]

I (13140) RC_CT: AVRC conn_state event: state 1, [5c:03:39:3e:be:0a]

I (13140) RC_CT: AVRC remote features 259, TG features c1

I (13180) RC_CT: remote rn_cap: count 7, bitmask 0x1e26

I (13360) RC_CT: AVRC metadata rsp: attribute id 0x1,

I (13360) RC_CT: AVRC metadata rsp: attribute id 0x2,

I (13360) RC_CT: AVRC metadata rsp: attribute id 0x4,

I (13360) RC_CT: AVRC metadata rsp: attribute id 0x20,

W (16190) BT_HCI: hci cmd send: sniff: hdl 0x80, intv(400 800)

W (16220) BT_HCI: hcif mode change: hdl 0x80, mode 2, intv 800, status 0x0

I (16220) BT_AV: ESP_BT_GAP_MODE_CHG_EVT mode: 2

I (26770) BT_LOG: bta_av_link_role_ok hndl:x41 role:1 conn_audio:x1 bits:1
features:x824b
```

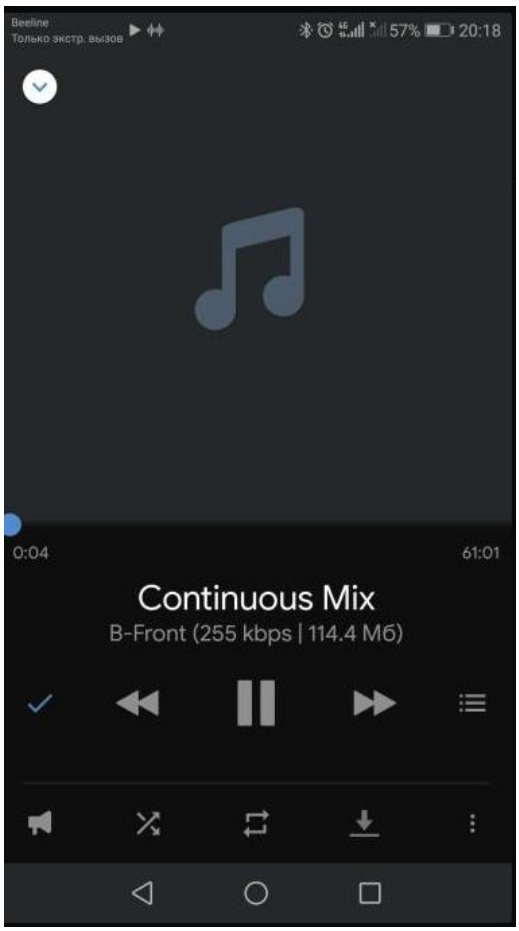

Рисунок 54 – Включение музыки на телефоне

Пример логов при включении музыки:

```
I (52240) BT_AV: A2DP audio state: Started
I (52240) RC_CT: AVRC event notification: 2
I (52250) RC_CT: AVRC event notification: 5
I (52250) BT_AV: Play position changed: 201-ms
I (52290) RC_CT: AVRC metadata rsp: attribute id 0x1, Continuous Mix
I (52290) RC_CT: AVRC metadata rsp: attribute id 0x2, B-Front
I (52290) RC_CT: AVRC metadata rsp: attribute id 0x4,
I (52300) RC_CT: AVRC metadata rsp: attribute id 0x20,
I (54360) BT_AV: Audio packet count: 100
I (56360) BT_AV: Audio packet count: 200
I (58360) BT_AV: Audio packet count: 300
```

Пример логов при остановке музыки:

```
I (58780) RC_CT: AVRC event notification: 5
I (58780) BT_AV: Play position changed: 6123-ms
I (59000) BT_AV: A2DP audio state: Suspended
I (59030) RC_CT: AVRC event notification: 1
```

```
I (59030) BT_AV: Playback status changed: 0x2
W (62190) BT_HCI: hci cmd send: sniff: hdl 0x80, intv(30 54)
W (62220) BT_HCI: hcif mode change: hdl 0x80, mode 2, intv 54, status 0x0
I (62220) BT_AV: ESP_BT_GAP_MODE_CHG_EVT mode: 2
```

Пример логов при отключении от устройства:

```
W (62250) BT_HCI: hcif mode change: hdl 0x80, mode 0, intv 0, status 0x0
I (62250) BT_AV: ESP_BT_GAP_MODE_CHG_EVT mode: 0
W (65450) BT_HCI: hci cmd send: sniff: hdl 0x80, intv(30 54)
W (65490) BT_HCI: hcif mode change: hdl 0x80, mode 2, intv 54, status 0x0
I (65490) BT_AV: ESP_BT_GAP_MODE_CHG_EVT mode: 2
W (79900) BT_HCI: hcif mode change: hdl 0x80, mode 0, intv 0, status 0x0
I (79900) BT_AV: ESP_BT_GAP_MODE_CHG_EVT mode: 0
W (79950) BT_HCI: hci cmd send: sniff: hdl 0x80, intv(30 54)
I (79950) RC_CT: AVRC conn_state event: state 0, [00:00:00:00:00:00]
I (79950) BT_AV: A2DP connection state: Disconnected, [5c:03:39:3e:be:0a]
I (79960) I2S: DMA queue destroyed
W (80010) BT_HCI: hcif mode change: hdl 0x80, mode 2, intv 54, status 0x0
I (80010) BT_AV: ESP_BT_GAP_MODE_CHG_EVT mode: 2
W (80080) BT_HCI: hcif disc complete: hdl 0x80, rsn 0x13
```

ЗАКЛЮЧЕНИЕ

Была достигнута цель курсового проекта - приобретение навыков в области проектирования, разработки и эксплуатации систем малой автоматизации, простроенных на базе микроконтроллерных плат семейства ESP-32.

Также, при выполнении курсовой работы были решены задачи курсового проекта и получены следующие результаты:

1) изучены функциональные особенности отладочной платы TTGO;

2) описан состав и функциональное назначение компонентов, интерфейсов платы TTGO;

3) написаны программы для работы с компонентами, интерфейсами платы и аудиокодеком.

Таким образом, плата TTGO является хорошим инструментом для отладки аудио приложений.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Компания Espressif. Обзор продукции. URL: https://wireless-e.ru/radiomoduli/espressif (дата обращения 14.12.2022)

2. Микроконтроллер ESP32 и проекты Arduino. URL: https://dzen.ru/a/Xyoe2iJJzWWqGcvK (дата обращения 14.12.2022)

3. Протокол I2S в ESP32: принципы работы, основы программирования. URL: https://microkontroller.ru/esp32-projects/protokol-i2s-v-esp32-princzipy-raboty-osnovy-programmirovaniya (дата обращения 14.12.2022)

4. Структура чипа ESP32 и работа его основных блоков. URL: https://wireless-e.ru/radiomoduli/esp32/ (дата обращения 14.12.2022)

5. Шина I2C в микроконтроллере ESP32. URL: http://digitrode.ru/computing-devices/mcu_cpu/2297-esp32-i-interfeys-i2c.html (дата обращения 14.12.2022)

6. Подключение датчика MPU9250 к Arduino. URL: https://microkontroller.ru/arduino-projects/podklyuchenie-datchika-mpu9250-k-arduino (дата обращения 15.12.2022)

7. I2S - аудиовоспроизведение и запись. URL: https://russianblogs.com/article/1824383095/ (дата обращения 15.12.2022)

8. Подключение адресной цветной (RGB) светодиодной ленты Neopixel к Arduino. URL: https://microkontroller.ru/arduino-projects/podklyuchenie-adresnoj-czvetnoj-rgb-svetodiodnoj-lenty-neopixel-k-arduino (дата обращения 15.12.2022)

9. Среда разработки Arduino. URL: https://arduino.ru/Arduino_environment (дата обращения 15.12.2022)

10. Установка ESP32 в Arduino IDE (руководство для ОС Windows). URL: https://wiki.iarduino.ru/page/esp32-windows (дата обращения 15.12.2022)

11. PlatformIO и Visual Studio Code для программирования Arduino, ESP32, Maixduino и не только. URL: https://micro-pi.ru/platformio-visual-studio-code-arduino (дата обращения 16.12.2022)

12. ESP32 Урок 2. Настраиваем toolchain и пишем свою прошивку в Espressif IDE. URL: https://narodstream.ru/esp32-urok-2-nastraivaem-toolchain-i-pishem-svoyu-proshivku-v-ide-es (дата обращения 16.12.2022)

ПРИЛОЖЕНИЕ А

```
#include "FS.h"
#include "SD.h"
#include "SPI.h"
#define SCK  14
#define MISO  2
#define MOSI  15
#define CS  13
void listDir(fs::FS &fs, const char * dirname, uint8_t levels)
{
  Serial.printf("Listing directory: %s\n", dirname);
  File root = fs.open(dirname);
  if(!root){
    Serial.println("Failed to open directory");
    return;
  }
  if(!root.isDirectory()){
    Serial.println("Not a directory");
    return;
  }
  File file = root.openNextFile();
  while(file){
    if(file.isDirectory()){
      Serial.print("  DIR : ");
      Serial.println(file.name());
      if(levels){
        listDir(fs, file.name(), levels -1);
      }
    }
else
{
      Serial.print("  FILE: ");
      Serial.print(file.name());
      Serial.print("  SIZE: ");
      Serial.println(file.size());
```

```
    }
    file = root.openNextFile();
  }
}
void createDir(fs::FS &fs, const char * path){
  Serial.printf("Creating Dir: %s\n", path);
  if(fs.mkdir(path)){
    Serial.println("Dir created");
  } else {
    Serial.println("mkdir failed");
  }
}
void removeDir(fs::FS &fs, const char * path){
  Serial.printf("Removing Dir: %s\n", path);
  if(fs.rmdir(path)){
    Serial.println("Dir removed");
  } else {
    Serial.println("rmdir failed");
  }
}
void readFile(fs::FS &fs, const char * path){
  Serial.printf("Reading file: %s\n", path);

  File file = fs.open(path);
  if(!file){
    Serial.println("Failed to open file for reading");
    return;
  }
  Serial.print("Read from file: ");
  while(file.available()){
    Serial.write(file.read());
  }
  file.close();
}

void writeFile(fs::FS &fs, const char * path, const char * message){
```

```
  Serial.printf("Writing file: %s\n", path);
  File file = fs.open(path, FILE_WRITE);
  if(!file){
    Serial.println("Failed to open file for writing");
    return;
  }
  if(file.print(message)){
    Serial.println("File written");
  } else {
    Serial.println("Write failed");
  }
  file.close();
}
void appendFile(fs::FS &fs, const char * path, const char * message){
  Serial.printf("Appending to file: %s\n", path);
  File file = fs.open(path, FILE_APPEND);
  if(!file){
    Serial.println("Failed to open file for appending");
    return;
  }
  if(file.print(message)){
    Serial.println("Message appended");
  } else {
    Serial.println("Append failed");
  }
  file.close();
}
void renameFile(fs::FS &fs, const char * path1, const char * path2){
  Serial.printf("Renaming file %s to %s\n", path1, path2);
  if (fs.rename(path1, path2)) {
    Serial.println("File renamed");
  } else {
    Serial.println("Rename failed");
  }
}
void deleteFile(fs::FS &fs, const char * path){
```

```
  Serial.printf("Deleting file: %s\n", path);
  if(fs.remove(path)){
    Serial.println("File deleted");
  } else {
    Serial.println("Delete failed");
  }
}
void testFileIO(fs::FS &fs, const char * path){
  File file = fs.open(path);
  static uint8_t buf[512];
  size_t len = 0;
  uint32_t start = millis();
  uint32_t end = start;
  if(file){
    len = file.size();
    size_t flen = len;
    start = millis();
    while(len){
      size_t toRead = len;
      if(toRead > 512){
        toRead = 512;
      }
      file.read(buf, toRead);
      len -= toRead;
    }
    end = millis() - start;
    Serial.printf("%u bytes read for %u ms\n", flen, end);
    file.close();
  } else {
    Serial.println("Failed to open file for reading");
  }
  file = fs.open(path, FILE_WRITE);
  if(!file){
    Serial.println("Failed to open file for writing");
    return;
  }
```

```
  size_t i;
  start = millis();
  for(i=0; i<2048; i++){
    file.write(buf, 512);
  }
  end = millis() - start;
  Serial.printf("%u bytes written for %u ms\n", 2048 * 512, end);
  file.close();
}
void setup()
{
  Serial.begin(115200);
  SPIClass spi = SPIClass(VSPI);
  spi.begin(SCK, MISO, MOSI, CS);
  if (!SD.begin(CS,spi,80000000)) {
    Serial.println("Card Mount Failed");
    return;
  }
  uint8_t cardType = SD.cardType();
  if(cardType == CARD_NONE){
    Serial.println("No SD card attached");
    return;
  }
  Serial.print("SD Card Type: ");
  if(cardType == CARD_MMC){
    Serial.println("MMC");
  } else if(cardType == CARD_SD){
    Serial.println("SDSC");
  } else if(cardType == CARD_SDHC){
    Serial.println("SDHC");
  } else {
    Serial.println("UNKNOWN");
  }
  uint64_t cardSize = SD.cardSize() / (1024 * 1024);
  Serial.printf("SD Card Size: %lluMB\n", cardSize);
  listDir(SD, "/", 0);
```

```
  createDir(SD, "/mydir");
  listDir(SD, "/", 0);
  removeDir(SD, "/mydir");
  listDir(SD, "/", 2);
  writeFile(SD, "/hello.txt", "Hello ");
  appendFile(SD, "/hello.txt", "World!\n");
  readFile(SD, "/hello.txt");
  deleteFile(SD, "/foo.txt");
  renameFile(SD, "/hello.txt", "/foo.txt");
  readFile(SD, "/foo.txt");
  testFileIO(SD, "/test.txt");
  Serial.printf("Total space: %lluMB\n", SD.totalBytes() / (1024 * 1024));
  Serial.printf("Used space: %lluMB\n", SD.usedBytes() / (1024 * 1024));
}
void loop()
{
}
```

ПРИЛОЖЕНИЕ Б

```
#include <Adafruit_NeoPixel.h>
#ifdef __AVR__
#include <avr/power.h>
#endif
#define PIN 22
#define NUM_LEDS 60
#define BRIGHTNESS 50
#define NUMPIXELS      20

Adafruit_NeoPixel strip = Adafruit_NeoPixel(NUM_LEDS, PIN, NEO_GRBW +
NEO_KHZ800);
Adafruit_NeoPixel pixels = Adafruit_NeoPixel(NUMPIXELS, PIN, NEO_GRB +
NEO_KHZ800);
void setup()
{
  strip.setBrightness(BRIGHTNESS);
  strip.begin();
  pixels.begin();
  strip.show();
  Serial.begin(115200);
}

void loop()
{
  if (Serial.available() > 0)
  {
      int sym=Serial.parseInt();
       switch (sym)
       {
         case 1:
         colorWipe(strip.Color(0,255,0), 50); // Red
         break;
         case 2:
         colorWipe(strip.Color(255, 0, 0), 50);  // Green
         break;
```

```
        case 3:
        colorWipe(strip.Color(0, 0, 255), 50); // BLUE
        break;
        case 4:
        colorWipe(strip.Color(0, 0, 0),1);
        colorWipes(strip.Color(0, 0, 0, 0),1);
        colorWipes(strip.Color(0, 255, 0),50); // RGB
        break;
        case 5:
        // Чистка
        colorWipe(strip.Color(0, 0, 0),1);
        colorWipes(strip.Color(0, 0, 0, 0),1);
        break;
      }
  }
}

void colorWipe(uint32_t c, uint8_t wait)
{
  for(uint16_t i=0; i<strip.numPixels(); i++)
  {
    pixels.setPixelColor(i,c);
    pixels.show();
    delay(wait);
  }
}
void colorWipes(uint32_t c, uint8_t wait)
 {

  for(uint16_t i=0; i<strip.numPixels(); i++)
  {
    strip.setPixelColor(i,c);
    strip.show();
    delay(wait);
  }
 }
```

ПРИЛОЖЕНИЕ В

```
#include <MPU9250_asukiaaa.h>
#define SDA_PIN 19
#define SCL_PIN 18

MPU9250_asukiaaa mySensor(MPU9250_ADDRESS_AD0_HIGH);
float aX, aY, aZ, aSqrt, gX, gY, gZ, mDirection, mX, mY, mZ;
   uint8_t sensorId;
  int result;
void setup()
{
  Serial.begin(115200);
  pinMode(36, INPUT_PULLUP);
  pinMode(34, INPUT_PULLUP);
  pinMode(39, INPUT_PULLUP);
  while(!Serial);
  Serial.println("started");
  Wire.begin(SDA_PIN, SCL_PIN);
  mySensor.setWire(&Wire);
  mySensor.beginAccel();
  mySensor.beginGyro();
  mySensor.beginMag();

  result = mySensor.readId(&sensorId);
  if (result == 0) Serial.println("sensorId: " + String(sensorId));
  else Serial.println("Cannot read sensorId " + String(result));

}

bool flag36 = false;
bool flag34 = false;
bool flag39 = false;

void loop()
{
```

```
//////Gyro
  bool btnState36 = !digitalRead(36);
  if (btnState36 && !flag36) // обработчик нажатия
  {
    flag36 = true;
    flag39 = false;
    flag34 = false;
    Serial.println("press Gyro");
  }

  if (!btnState36 && flag36) // обработчик отпускания
  {
  result = mySensor.accelUpdate();
  if (result == 0)
   {
    aX = mySensor.accelX();
    aY = mySensor.accelY();
    aZ = mySensor.accelZ();
    Serial.println("GyrolX: " + String(aX));
    Serial.println("GyrolY: " + String(aY));
    Serial.println("GyrolZ: " + String(aZ));
   }
   else Serial.println("Cannod read Gyro values " + String(result));

   Serial.println("");
   delay(200);
  }
//////Assel


  bool btnState34 = !digitalRead(34);
  if (btnState34 && !flag34) // обработчик нажатия
  {
    flag36 = false;
    flag39 = false;
```

```
    Serial.println("stop");
    flag34 = true;
  }

if (!btnState34 && flag34) // обработчик отпускания
  {
  result = mySensor.gyroUpdate();
  if (result == 0)
   {
    gX = mySensor.gyroX();
    gY = mySensor.gyroY();
    gZ = mySensor.gyroZ();
    Serial.println("AsselX: " + String(gX));
    Serial.println("AsselY: " + String(gY));
    Serial.println("AsselZ: " + String(gZ));
   }
   else Serial.println("Cannot read Assel values " + String(result));

   Serial.println("");
   delay(200);
  }
///////////Mag


bool btnState39 = !digitalRead(39);
  if (btnState39 && !flag39) // обработчик нажатия
  {
    flag36 = false;
    flag34 = false;
    Serial.println("stop");
    flag39 = true;
  }

if (!btnState39 && flag39) // обработчик отпускания
  {
 result = mySensor.magUpdate();
```

```
  if (result == 0)
  {
    mX = mySensor.magX();
    mY = mySensor.magY();
    mZ = mySensor.magZ();
    mDirection = mySensor.magHorizDirection();
    Serial.println("magX: " + String(mX));
    Serial.println("maxY: " + String(mY));
    Serial.println("magZ: " + String(mZ));
    Serial.println("horizontal direction: " + String(mDirection));
  }
  else Serial.println("Cannot read mag values " + String(result));
  Serial.println("");
  delay(200);
  }


}
```

ПРИЛОЖЕНИЕ Г

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

#include "WM8978.h"
#include "FaustSawtooth.h"
FaustSawtooth faustSawtooth(48000,8);
WM8978 wm8978;
void setup() {
  //WM8978 wm8978;
  wm8978.init();
  wm8978.addaCfg(1,1);
  wm8978.inputCfg(1,0,0);
  wm8978.outputCfg(1,0);
  wm8978.micGain(30);
  wm8978.auxGain(0);
  wm8978.lineinGain(0);
  wm8978.spkVolSet(0);
  wm8978.hpVolSet(10,10);
  wm8978.i2sCfg(2,0);
  faustSawtooth.start();
}
void loop()
{
  wm8978.hpVolSet(10,10);
  faustSawtooth.setParamValue("freq",rand()%(2000-50 + 1) + 50);
  delay(1000);
  wm8978.hpVolSet(10,0);
    faustSawtooth.setParamValue("freq",rand()%(2000-50 + 1) + 50);
  delay(1000);
    wm8978.hpVolSet(0,10);
    faustSawtooth.setParamValue("freq",rand()%(2000-50 + 1) + 50);
  delay(1000);
}
```

ПРИЛОЖЕНИЕ Д

```
#include "main.h"
static const char *TAG = "main";
void app_main(void)
{
      //Вифи инфо
      wifi_ap_record_t info;
      //Светодиодик
      gpio_reset_pin(CONFIG_LED_GPIO);
      gpio_set_direction(CONFIG_LED_GPIO, GPIO_MODE_OUTPUT);
      gpio_set_level(CONFIG_LED_GPIO, 0);
      //Память NVS
        esp_err_t ret = nvs_flash_init();
        if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == 
ESP_ERR_NVS_NEW_VERSION_FOUND) {
          ret = nvs_flash_erase();
          ESP_LOGI(TAG, "nvs_flash_erase: 0x%04x", ret);
          ret = nvs_flash_init();
          ESP_LOGI(TAG, "nvs_flash_init: 0x%04x", ret);
        }
        ESP_LOGI(TAG, "nvs_flash_init: 0x%04x", ret);
        //инициализация
        wifi_init_sta();
        xTaskCreate(udp_task, "udp_task", 4096, NULL, 5, NULL);
    while (true) {
      ret = esp_wifi_sta_get_ap_info(&info);
      ESP_LOGI(TAG, "esp_wifi_sta_get_ap_info: 0x%04x", ret);
      if(ret==0) ESP_LOGI(TAG, "SSID: %s", info.ssid);
      else
      {
            gpio_set_level(CONFIG_LED_GPIO, 0);
            wifi_init_sta();
          }
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}
```

ПРИЛОЖЕНИЕ Е

```
#include <WM8978.h>
#include <Audio.h>

// T-Audio 1.6 WM8978 I2C pins.
#define I2C_SDA     19
#define I2C_SCL     18

// T-Audio 1.6 WM8978 I2S pins.
#define I2S_BCK     33
#define I2S_WS      25
#define I2S_DOUT    26
#define I2S_DIN     27
#define I2S_MCLKPIN  0

WM8978 dac;
Audio audio;

void setup() {

  // Setup wm8978 I2C interface.
  if (!dac.begin(I2C_SDA, I2C_SCL)) {
    ESP_LOGE(TAG, "Error setting up dac: System halted.");
    while (1) delay(100);
  }

  // Select I2S pins
  audio.setPinout(I2S_BCK, I2S_WS, I2S_DOUT);
  audio.i2s_mclk_pin_select(I2S_MCLKPIN);

  // WiFi Settings here.
  WiFi.begin("Anime", "12345678");
  while (!WiFi.isConnected()) {
    delay(10);
  }
```

```
  ESP_LOGI(TAG, "Connected. Starting MP3...");

  // Enter your Icecast station URL here.

  audio.connecttohost("https://str.pcradio.ru/energy_spb-hi");

  // Volume control.

  dac.setSPKvol(20); // Change volume here for board speaker output (Max 63).

  dac.setHPvol(52, 52); // Change volume here for headphone jack left, right
channel.

}

void loop()

{

  // Start the stream.

  audio.loop();

}
```

ПРИЛОЖЕНИЕ Ж

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "nvs.h"
#include "nvs_flash.h"
#include "esp_system.h"
#include "esp_log.h"
#include "esp_bt.h"
#include "bt_app_core.h"
#include "bt_app_av.h"
#include "esp_bt_main.h"
#include "esp_bt_device.h"
#include "esp_gap_bt_api.h"
#include "esp_a2dp_api.h"
#include "esp_avrc_api.h"
#include "driver/i2s.h"

/* device name */
#define LOCAL_DEVICE_NAME    "M"

/* event for stack up */
enum {
    BT_APP_EVT_STACK_UP = 0,
};

/*********************************
 * STATIC FUNCTION DECLARATIONS
 ********************************/
/* GAP callback function */
static void bt_app_gap_cb(esp_bt_gap_cb_event_t event, esp_bt_gap_cb_param_t
*param);
/* handler for bluetooth stack enabled events */
```

```
static void bt_av_hdl_stack_evt(uint16_t event, void *p_param);


/******************************
 * STATIC FUNCTION DEFINITIONS
 ******************************/


static void bt_app_gap_cb(esp_bt_gap_cb_event_t event, esp_bt_gap_cb_param_t
*param)
{
    switch (event) {
    /* when authentication completed, this event comes */
    case ESP_BT_GAP_AUTH_CMPL_EVT: {
        if (param->auth_cmpl.stat == ESP_BT_STATUS_SUCCESS) {
            ESP_LOGI(BT_AV_TAG, "authentication success: %s", param-
>auth_cmpl.device_name);
            esp_log_buffer_hex(BT_AV_TAG, param->auth_cmpl.bda,
ESP_BD_ADDR_LEN);
        } else {
            ESP_LOGE(BT_AV_TAG, "authentication failed, status: %d", param-
>auth_cmpl.stat);
        }
        break;
    }


#if (CONFIG_BT_SSP_ENABLED == true)
    /* when Security Simple Pairing user confirmation requested, this event
comes */
    case ESP_BT_GAP_CFM_REQ_EVT:
        ESP_LOGI(BT_AV_TAG, "ESP_BT_GAP_CFM_REQ_EVT Please compare the
numeric value: %d", param->cfm_req.num_val);
        esp_bt_gap_ssp_confirm_reply(param->cfm_req.bda, true);
        break;
    /* when Security Simple Pairing passkey notified, this event comes */
    case ESP_BT_GAP_KEY_NOTIF_EVT:
        ESP_LOGI(BT_AV_TAG, "ESP_BT_GAP_KEY_NOTIF_EVT passkey: %d", param-
>key_notif.passkey);
        break;
    /* when Security Simple Pairing passkey requested, this event comes */
    case ESP_BT_GAP_KEY_REQ_EVT:
```

```
        ESP_LOGI(BT_AV_TAG, "ESP_BT_GAP_KEY_REQ_EVT Please enter passkey!");
        break;
#endif

    /* when GAP mode changed, this event comes */
    case ESP_BT_GAP_MODE_CHG_EVT:
        ESP_LOGI(BT_AV_TAG,   "ESP_BT_GAP_MODE_CHG_EVT   mode:   %d",   param-
>mode_chg.mode);
        break;
    /* others */
    default: {
        ESP_LOGI(BT_AV_TAG, "event: %d", event);
        break;
    }
    }
}

static void bt_av_hdl_stack_evt(uint16_t event, void *p_param)
{
    ESP_LOGD(BT_AV_TAG, "%s event: %d", __func__, event);

    switch (event) {
    /* when do the stack up, this event comes */
    case BT_APP_EVT_STACK_UP: {
        esp_bt_dev_set_device_name(LOCAL_DEVICE_NAME);
        esp_bt_gap_register_callback(bt_app_gap_cb);

        assert(esp_avrc_ct_init() == ESP_OK);
        esp_avrc_ct_register_callback(bt_app_rc_ct_cb);
        assert(esp_avrc_tg_init() == ESP_OK);
        esp_avrc_tg_register_callback(bt_app_rc_tg_cb);
        esp_avrc_rn_evt_cap_mask_t evt_set = {0};
        esp_avrc_rn_evt_bit_mask_operation(ESP_AVRC_BIT_MASK_OP_SET,
&evt_set, ESP_AVRC_RN_VOLUME_CHANGE);
        assert(esp_avrc_tg_set_rn_evt_cap(&evt_set) == ESP_OK);

        assert(esp_a2d_sink_init() == ESP_OK);
```

```
        esp_a2d_register_callback(&bt_app_a2d_cb);
        esp_a2d_sink_register_data_callback(bt_app_a2d_data_cb);

        /* set discoverable and connectable mode, wait to be connected */
        esp_bt_gap_set_scan_mode(ESP_BT_CONNECTABLE,
ESP_BT_GENERAL_DISCOVERABLE);
        break;
    }
    /* others */
    default:
        ESP_LOGE(BT_AV_TAG, "%s unhandled event: %d", __func__, event);
        break;
    }
}

/*******************************
 * MAIN ENTRY POINT
 *******************************/

void app_main(void)
{
    /* initialize NVS — it is used to store PHY calibration data */
    esp_err_t err = nvs_flash_init();
    if      (err      ==      ESP_ERR_NVS_NO_FREE_PAGES      ||      err      ==
ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        err = nvs_flash_init();
    }
    ESP_ERROR_CHECK(err);
    /*
     * This example only uses the functions of Classical Bluetooth.
     * So release the controller memory for Bluetooth Low Energy.
     */
    ESP_ERROR_CHECK(esp_bt_controller_mem_release(ESP_BT_MODE_BLE));

    esp_bt_controller_config_t bt_cfg = BT_CONTROLLER_INIT_CONFIG_DEFAULT();
    if ((err = esp_bt_controller_init(&bt_cfg)) != ESP_OK) {
```

```
        ESP_LOGE(BT_AV_TAG,   "%s   initialize   controller   failed:   %s\n",
__func__, esp_err_to_name(err));

        return;

    }

    if ((err = esp_bt_controller_enable(ESP_BT_MODE_CLASSIC_BT)) != ESP_OK) {

        ESP_LOGE(BT_AV_TAG,  "%s  enable  controller  failed:  %s\n",  __func__,
esp_err_to_name(err));

        return;

    }

    if ((err = esp_bluedroid_init()) != ESP_OK) {

        ESP_LOGE(BT_AV_TAG, "%s initialize bluedroid failed: %s\n", __func__,
esp_err_to_name(err));

        return;

    }

    if ((err = esp_bluedroid_enable()) != ESP_OK) {

        ESP_LOGE(BT_AV_TAG,  "%s  enable  bluedroid  failed:  %s\n",  __func__,
esp_err_to_name(err));

        return;

    }

#if (CONFIG_BT_SSP_ENABLED == true)

    /* set default parameters for Secure Simple Pairing */

    esp_bt_sp_param_t param_type = ESP_BT_SP_IOCAP_MODE;

    esp_bt_io_cap_t iocap = ESP_BT_IO_CAP_IO;

    esp_bt_gap_set_security_param(param_type, &iocap, sizeof(uint8_t));

#endif

    /* set default parameters for Legacy Pairing (use fixed pin code 1234) */

    esp_bt_pin_type_t pin_type = ESP_BT_PIN_TYPE_FIXED;

    esp_bt_pin_code_t pin_code;

    pin_code[0] = '1';

    pin_code[1] = '2';

    pin_code[2] = '3';

    pin_code[3] = '4';

    esp_bt_gap_set_pin(pin_type, 4, pin_code);

    bt_app_task_start_up();

    /* bluetooth device name, connection mode and profile set up */

    bt_app_work_dispatch(bt_av_hdl_stack_evt,  BT_APP_EVT_STACK_UP,  NULL,  0,
NULL);

}
```